Angular NgRx Learning Series

Topics 51 — 59

Consolidated | Modern Angular | June 2026

NgRx Core Concepts • Effects • Selectors In Depth • Entity • Component Store • SignalStore • Router Store • Testing • Best Practices

TOPIC 51 — NgRx Core Concepts (Store, Actions, Reducers, Selectors)

Consolidated | Difficulty: Intermediate | Relevance: Critical

---

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — The individual pieces (actions, reducers, selectors, store) are each simple; the difficulty is understanding why they are kept deliberately separate and how unidirectional data flow makes the whole system more predictable than the sum of its parts.

Angular Relevance: Critical — NgRx is the dominant state management solution in professional Angular teams; understanding its core mental model is a prerequisite for every topic from Topic 52 onward, and misunderstanding it is the single most common source of unmaintainable Angular codebases at scale.

Section 1 — Explanation

The Bank Ledger Analogy

Imagine a bank. The bank's vault holds the current balance (the Store — one authoritative state object). You cannot walk into the vault and change the number yourself. Instead, you fill out a deposit slip or withdrawal form (an Action — a plain description of what you want to happen, with no logic in it). A teller takes your form, reads the current balance, applies the transaction according to strict rules, and writes a new balance in the ledger (the Reducer — a pure function that takes current state + action and returns new state). At any time, you can ask the bank for a statement (a Selector — a memoised query that reads a specific slice of the ledger without exposing the raw vault).

Critically: nobody ever reaches into the vault directly. Every change goes through the same controlled pathway. This is what makes the system auditable, replayable, and testable — not because the pieces are clever, but because the rules are strict.

Technical Explanation

NgRx implements the Redux pattern for Angular using RxJS. The four core pieces:

- Store — A single BehaviorSubject-backed object holding your entire application state tree. Components and services never mutate it directly. They read it through selectors and write to it by dispatching actions.

- Actions — Plain TypeScript objects with a mandatory type string property. Created with createAction(). They are descriptions of events, not commands — [User Page] Load Users Clicked not loadUsers(). This naming discipline matters at scale.

- Reducers — Pure functions: (currentState, action) => newState. Created with createReducer() + on(). They must be synchronous, have no side effects, and always return a new state object — never mutate the existing one. They are the only place state changes.

- Selectors — Pure functions that derive data from the store. Created with createSelector(). They are memoised — if the input slice hasn't changed (by reference), the projector function doesn't re-run. Components subscribe to selectors, not to raw store slices.

The Unidirectional Data Flow

```
Component dispatches Action ↓ Reducer computes new State ↓ Store emits new State ↓ Selector derives slice of State ↓ Component receives updated data ↓ (cycle repeats)
```

Side effects (HTTP calls, etc.) sit outside this cycle — they listen to actions, do async work, then dispatch new actions back into the cycle. That is Topic 52.

Piece	Created with	Role	Pure?
Store	provideStore(reducers)	Holds all state	N/A
Action	createAction(type, props)	Describes an event	Yes — no logic
Reducer	createReducer(initial, on(...))	Computes new state	Yes — must be
Selector	createSelector(inputs, projector)	Reads/derives state	Yes — memoised

Section 2 — Connection to Prior Concepts

Directly builds on:

- Topic 44 — State Management Patterns: NgRx is the industrial-scale implementation of exactly the store pattern you built manually in Topic 44 — private state, public read-only projections, write-only methods. NgRx formalises each of those concepts into its own named piece.

- Topic 20 — RxJS Subscription Management and Topic 46 — RxJS Operators in Depth: The Store is a stream — store.select(selector) returns an Observable. Everything you know about subscription cleanup, takeUntilDestroyed, and the async pipe applies directly.

- Topic 22 — Async Pipe: Selectors return Observables — the async pipe is the standard way to consume them in templates, giving you automatic subscription management and OnPush compatibility.

- Topic 24 — OnPush Change Detection: NgRx selectors emit only when their memoised output changes — this pairs perfectly with OnPush, because Angular only needs to check a component when its selector actually produces a new value.

- Topic 14 — readonly & Immutability: Reducers must return new state objects, never mutate existing ones — the immutability rules from Topic 14 are not optional suggestions in NgRx, they are correctness requirements.

Bridging note: In Topic 44 you built a store service with a BehaviorSubject, private state, and public write methods. NgRx is that pattern taken to its logical extreme — each concern is pulled into its own dedicated, independently testable file. The BehaviorSubject becomes the Store, the write methods become Actions + Reducers, the public read-only projection becomes a Selector.

The pattern is identical; the discipline is enforced by convention and tooling rather than by you.

Most directly extends: Topic 44 — State Management Patterns.

Section 3 — Code Example

Plain TS — Core concept in isolation (no Angular)

```
// Understanding the reducer pattern before adding NgRx interface CounterState { count: number; } const initialState: CounterState = { count: 0 }; function counterReducer( state = initialState, action: { type: string; amount?: number } ): CounterState { switch (action.type) { case '[Counter] Increment': return { ...state, count: state.count + 1 }; case '[Counter] Add': return { ...state, count: state.count + (action.amount ?? 0) }; default: return state; } } const selectCount = (state: CounterState) => state.count; const state1 = counterReducer(undefined, { type: '@@INIT' }); // { count: 0 } const state2 = counterReducer(state1, { type: '[Counter] Increment' }); // { count: 1 } const state3 = counterReducer(state2, { type: '[Counter] Add', amount: 5 }); // { count: 6 } console.log(selectCount(state3)); // 6
```

Real Angular NgRx — Full feature slice

```
// users/store/users.actions.ts import { createAction, props } from '@ngrx/store'; import { User } from '../models/user.model'; export const loadUsersPageOpened = createAction('[Users Page] Page Opened'); export const userSearched = createAction('[Users Page] User Searched', props<{ query: string }>()); export const loadUsersSuccess = createAction('[Users API] Load Users Success', props<{ users: User[] }>()); export const loadUsersFailure = createAction('[Users API] Load Users Failure', props<{ error: string }>());
```

```
// users/store/users.reducer.ts import { createReducer, on } from '@ngrx/store'; import { User } from '../models/user.model'; import * as UsersActions from './users.actions'; export interface UsersState { users: User[]; loading: boolean; error: string | null; searchQuery: string; } const initialState: UsersState = { users: [], loading: false, error: null, searchQuery: '' }; export const usersReducer = createReducer( initialState, on(UsersActions.loadUsersPageOpened, state => ({ ...state, loading: true, error: null })), on(UsersActions.userSearched, (state, { query }) => ({ ...state, searchQuery: query })), on(UsersActions.loadUsersSuccess, (state, { users }) => ({ ...state, users, loading: false, error: null })), on(UsersActions.loadUsersFailure, (state, { error }) => ({ ...state, loading: false, error })) );
```

```
// users/store/users.selectors.ts import { createFeatureSelector, createSelector } from '@ngrx/store'; import { UsersState } from './users.reducer'; const selectUsersState = createFeatureSelector<UsersState>('users'); export const selectAllUsers = createSelector(selectUsersState, state => state.users); export const selectUsersLoading = createSelector(selectUsersState, state => state.loading); export const selectUsersError = createSelector(selectUsersState, state => state.error); export const selectSearchQuery = createSelector(selectUsersState, state => state.searchQuery); export const selectFilteredUsers = createSelector( selectAllUsers, selectSearchQuery, (users, query) => { if (!query.trim()) return users; const lower = query.toLowerCase(); return users.filter(u => u.name.toLowerCase().includes(lower) || u.email.toLowerCase().includes(lower) ); } ); export const selectUserById = (id: number) => createSelector( selectAllUsers, users => users.find(u => u.id === id) ?? null );
```

```
// app.config.ts import { ApplicationConfig } from '@angular/core'; import { provideStore } from '@ngrx/store'; import { provideStoreDevtools } from '@ngrx/store-devtools'; export const appConfig: ApplicationConfig = { providers: [ provideStore({}), provideStoreDevtools({ maxAge: 25, logOnly: false, autoPause: true }), ] }; // users/users.routes.ts import { Routes } from '@angular/router'; import { provideState } from '@ngrx/store'; import { usersReducer } from './store/users.reducer'; import { UsersPageComponent } from './users-page.component'; export const usersRoutes: Routes = [{ path: '', providers: [provideState('users', usersReducer)], component: UsersPageComponent, }];
```

```
// users/users-page.component.ts import { Component, inject, OnInit, ChangeDetectionStrategy
} from '@angular/core'; import { AsyncPipe } from '@angular/common'; import { Store } from
'@ngrx/store'; import * as UsersActions from './store/users.actions'; import {
selectFilteredUsers, selectUsersLoading, selectUsersError } from './store/users.selectors';
@Component({ selector: 'app-users-page', standalone: true, imports: [AsyncPipe],
changeDetection: ChangeDetectionStrategy.OnPush, template: ` <input #searchInput
(input)="onSearch(searchInput.value)" placeholder="Search users..." /> @if (loading$ | async)
{ <p>Loading...</p> } @if (error$ | async; as error) { <p class="error">{{ error }}</p> } @for
(user of filteredUsers$ | async; track user.id) { <div class="user-card"><h3>{{ user.name
}}</h3><p>{{ user.email }}</p></div> } ` }) export class UsersPageComponent implements OnInit
{ readonly #store = inject(Store); filteredUsers$ = this.#store.select(selectFilteredUsers);
loading$ = this.#store.select(selectUsersLoading); error$ =
this.#store.select(selectUsersError); ngOnInit(): void {
this.#store.dispatch(UsersActions.loadUsersPageOpened()); } onSearch(query: string): void {
this.#store.dispatch(UsersActions.userSearched({ query })); } }
```

Section 4 — Before & After Refactor

Before — Service-based state with direct mutation

```
// WRONG: State scattered across services, mutated directly @Injectable({ providedIn: 'root'
}) export class UsersService { users: User[] = []; // public mutable — anything can change
this loading = false; loadUsers(): void { this.loading = true;
this.http.get<User[]>('/api/users').subscribe(users => { this.users = users; // direct
mutation this.loading = false; }); } } // Component A mutates the service array directly
this.usersService.users.push(newUser); // no other component knows // Component B reads stale
data with no notification // OnPush components never update — array reference doesn't change
on push()
```

After — NgRx unidirectional flow

```
// CORRECT: All state changes go through actions -> reducers this.store.dispatch(userAdded({
user: newUser })); // Reducer handles it — pure, testable, single place on(userAdded, (state,
{ user }) => ({ ...state, users: [...state.users, user] // new reference — OnPush updates }))
// Selector emits the new value — all subscribers update simultaneously
```

Why: The new array reference from spread in the reducer triggers OnPush change detection in every component subscribed to selectAllUsers — simultaneously, consistently, with zero manual markForCheck() calls.

Section 5 — Common Mistakes

Mistake 1 — Dispatching actions that describe commands instead of events

```
// WRONG: Command-style action names export const fetchUsers = createAction('[Users] Fetch
Users'); export const setLoading = createAction('[Users] Set Loading', props<{ loading:
boolean }>()); export const updateUser = createAction('[Users] Update User', props<{ user:
User }>());
```

What it is: Naming actions as imperative commands instead of past-tense domain events.

Why it happens: Coming from imperative programming, actions feel like method calls.

How to avoid it: Name actions as events from a specific source — [Users Page] Page Opened, [Users API] Load Users Success. An action describes what happened; multiple reducers and effects can react to it independently.

Angular consequence: Command-style actions create tight coupling — one action becomes responsible for one specific mutation. Event-style actions allow multiple slices of state and multiple effects to react to the same event, which is the scalability advantage NgRx provides.

Mistake 2 — Mutating state inside a reducer

```
// WRONG: Direct mutation inside a reducer on(loadUsersSuccess, (state, { users }) => {
state.users = users; // MUTATION — modifies the existing object state.loading = false; return
state; // returning the same reference })
```

What it is: Modifying the existing state object instead of returning a new one.

Why it happens: It looks like it works — the values change. Developers from mutable patterns don't see the problem immediately.

How to avoid it: Always return a new object using spread: { ...state, users, loading: false }. Enable runtime checks: provideStore({}, { runtimeChecks: { strictStateImmutability: true } }).

Angular consequence: Selectors won't emit new values because the object reference hasn't changed. OnPush components freeze. DevTools shows the action dispatched but the state diff is empty.

Mistake 3 — Subscribing to the store manually without cleanup

```
// WRONG: Manual subscription without takeUntilDestroyed constructor() {
this.store.select(selectAllUsers).subscribe(users => { this.users = users; // no cleanup });
}
```

What it is: Manually subscribing to store.select() without using takeUntilDestroyed or the async pipe.

Why it happens: Manual subscription is familiar — the async pipe feels like extra syntax.

How to avoid it: Assign store.select() to a class property and consume with the async pipe. If you must subscribe manually, add takeUntilDestroyed() (Topic 27).

Angular consequence: Every component instance created and destroyed leaves a new active subscription — memory leaks and ghost state updates.

Section 6 — Do's and Don'ts

DO	DON'T
Name actions as past-tense events with a [Source] prefix	Name actions as commands (fetchData, setLoading)
Always spread state in reducers: { ...state, changed: value }	Mutate state properties directly inside reducers
Enable runtime immutability checks in development	Skip runtime checks and discover mutations as mysterious bugs
Use createFeatureSelector + createSelector for all state access	Access store inline: store.select(state => state['feature']['users'])

DO	DON'T
Register feature state lazily with provideState() in route providers	Register all feature reducers upfront in provideStore() — defeats lazy loading
Use ChangeDetectionStrategy.OnPush on every component using the store	Use default change detection — wastes render cycles
Install @ngrx/store-devtools from day one	Debug NgRx without DevTools — it's guesswork

Section 7 — Interview Prep

Question 1

"Explain the role of each piece in NgRx — Store, Action, Reducer, Selector — and why they are kept separate."

Strong answer covers:

- Store: single source of truth, read-only externally, updated only through reducers
- Actions: plain event descriptions, enable any part of the app to react to the same event
- Reducers: pure functions, the only place state changes, enabling time-travel and testability
- Selectors: memoised derived data, decouple component templates from state shape
- Separation enables independent testability of each piece and allows multiple reducers/effects to react to one action

Weak answers miss: Why separation matters — candidates describe what each piece does without explaining why the separation itself is the architectural benefit.

Level: Mid

Question 2

"What does pure function mean in the context of NgRx reducers, and what breaks if a reducer isn't pure?"

Strong answer covers:

- Pure: same inputs always produce same output, no side effects, no external state reads
- Mutation breaks selector memoisation — same reference = no emission = frozen UI
- Impure reducers (reading Date.now() or Math.random()) cause DevTools time-travel replay to diverge from original state
- Async operations in reducers break the synchronous state transition guarantee
- strictStateImmutability runtime check catches violations in development

Weak answers miss: The time-travel debugging consequence — most candidates mention OnPush but miss that the whole DevTools replay mechanism depends on reducer purity.

Level: Mid

Section 8 — Quick Reference Card

3-line definition: NgRx enforces a one-way cycle — components dispatch event descriptions (Actions), pure functions compute new state (Reducers), a single object holds all state (Store), and memoised queries derive component data (Selectors). Nobody touches the state directly. Every change is logged, replayable, and testable.

Syntax at a glance:

```
// Action export const userSearched = createAction('[Users Page] User Searched', props<{
query: string }>()); // Reducer export const usersReducer = createReducer( initialState,
on(UsersActions.userSearched, (state, { query }) => ({ ...state, searchQuery: query })) ); //
Selector export const selectFilteredUsers = createSelector(selectAllUsers, selectSearchQuery,
(users, query) => users.filter(u => u.name.includes(query)) ); // Component readonly #store =
inject(Store); users$ = this.#store.select(selectFilteredUsers);
this.#store.dispatch(UsersActions.userSearched({ query }));
```

When to use: State shared across multiple routes; side effects that need audit trails; teams larger than three people; complex state transitions.

When not to use: Feature-local state (use SignalStore or a Signal service); simple single-component state; small apps where boilerplate slows delivery.

The one rule that matters most: Never mutate state in a reducer — always return a new object with spread syntax. Mutation silently breaks selector memoisation, freezes OnPush components, and corrupts DevTools time-travel — all with no error message.

TOPIC 52 — NgRx Effects

Consolidated | Difficulty: Intermediate-Advanced | Relevance: Critical

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate-Advanced — The concept is straightforward (listen to actions, do async work, dispatch new actions), but the operator choice (switchMap vs concatMap vs exhaustMap) has silent, destructive consequences in production that are non-obvious until you know exactly what each one cancels.

Angular Relevance: Critical — Every real NgRx app has HTTP calls, navigation, analytics, and error handling that must live outside reducers; Effects are the only correct place for all of it, and getting the operator choice wrong is the most common source of data loss bugs in production NgRx applications.

Section 1 — Explanation

The Personal Assistant Analogy

Imagine your NgRx Store is a CEO. The CEO only does one thing: makes decisions based on information (reducers computing state). The CEO never picks up the phone, never emails clients, never fetches documents — that's beneath the role and would distract from decision-making.

So the CEO has a personal assistant (an Effect). The assistant listens in on all meetings (listens to dispatched actions), and whenever a relevant topic comes up — say, 'we need the Q3 report' — the assistant quietly leaves the room, makes the phone calls, fetches the document, and returns with either 'here's the report' (loadReportsSuccess) or 'the server was down' (loadReportsFailure). The CEO (reducer) then makes a decision based on that new information.

The assistant never changes the CEO's calendar directly. They always bring back a formal memo (dispatch a new action). This is the Effects contract.

Technical Explanation

An Effect is a class decorated with @Injectable that uses Actions — an Observable stream of every action dispatched in the app — combined with ofType() to filter for specific actions, followed by an RxJS operator chain that performs the side effect, and ends by dispatching a new action (or using { dispatch: false } for fire-and-forget effects like analytics).

The critical decision in every Effect is which flattening operator to use after ofType():

- switchMap — cancels the previous inner Observable when a new action arrives. Right for searches, navigation, any 'latest wins' scenario. Catastrophic for writes — cancels in-flight POST/PUT/DELETE on double-click.
- concatMap — queues actions, processes one at a time in strict order. Right for sequential writes where order matters.

- exhaustMap — ignores new actions while one is already in flight. Right for login — ignores double-clicks entirely until the current request completes.

- mergeMap — runs all concurrently with no cancellation. Right for independent parallel operations where order doesn't matter.

Operator	Behaviour	Use case	Never use for
switchMap	Cancels previous, uses latest	Search, autocomplete, navigation	POST/PUT/DELETE
concatMap	Queues, strict order	Sequential writes, ordered operations	High-frequency events
exhaustMap	Ignores new while busy	Login, form submit	Operations that must all complete
mergeMap	All concurrent, no cancel	Independent parallel reads	Ordered operations

Section 2 — Connection to Prior Concepts

Directly builds on:

- Topic 51 — NgRx Core Concepts: Effects complete the NgRx cycle — actions flow in, Effects react, new actions flow back out. You must understand the action/reducer/store cycle before Effects make sense.

- Topic 46 — RxJS Operators in Depth: The four flattening operators (switchMap, concatMap, exhaustMap, mergeMap) were covered in Topic 46 in the context of HTTP. Effects apply exactly the same operators with exactly the same cancellation behaviours — the stakes are just higher because the wrong choice silently drops writes.

- Topic 37 — HttpClient & API Integration: Effects are where HttpClient calls live in an NgRx app — the service layer you built in Topic 37 is called from inside Effects.

- Topic 18 — Error Handling: Every Effect that makes an HTTP call must catch errors with catchError and dispatch a failure action — an unhandled error in an Effect kills the Effect Observable permanently, silently disabling it for the rest of the session.

- Topic 38 — HTTP Interceptors: Interceptors still run for HTTP calls made inside Effects — understanding the middleware chain matters when debugging auth failures in Effects.

Bridging note: In Topic 46 you learned that switchMap cancels in-flight requests — great for search, dangerous for writes. In Topic 37 you put HTTP calls in services. Effects are the architectural layer that connects those two insights: they call your Topic 37 services using your Topic 46 operator knowledge, triggered by your Topic 51 actions. Effects are where everything comes together.

Most directly extends: Topic 46 — RxJS Operators in Depth (same operators, higher-stakes context).

Section 3 — Code Example

Plain TS — Core concept in isolation

```
import { Subject, of } from 'rxjs'; import { filter, switchMap, catchError, map } from 'rxjs';
const actions$ = new Subject<{ type: string; query?: string }>(); actions$.pipe(
filter(action => action.type === '[Search] Query Changed'), switchMap(action =>
fakeHttpSearch(action.query!).pipe( map(results => ({ type: '[Search API] Results Loaded',
results })), catchError(err => of({ type: '[Search API] Load Failed', error: err.message })) )
) ).subscribe(newAction => { actions$.next(newAction); });
```

Real Angular NgRx Effects

```
// ng add @ngrx/effects // users/store/users.effects.ts import { Injectable, inject } from
'@angular/core'; import { Actions, createEffect, ofType } from '@ngrx/effects'; import {
Store } from '@ngrx/store'; import { Router } from '@angular/router'; import { of } from
'rxjs'; import { switchMap, map, catchError, exhaustMap, tap, concatMap, withLatestFrom }
from 'rxjs'; import * as UsersActions from './users.actions'; import { UsersApiService } from
'../services/users-api.service'; @Injectable() export class UsersEffects { readonly #actions$
= inject(Actions); readonly #usersApi = inject(UsersApiService); readonly #router =
inject(Router); readonly #store = inject(Store); loadUsers$ = createEffect(() =>
this.#actions$.pipe( ofType(UsersActions.loadUsersPageOpened), switchMap(() =>
this.#usersApi.getUsers().pipe( map(users => UsersActions.loadUsersSuccess({ users })),
catchError(error => of(UsersActions.loadUsersFailure({ error: error.message })))  ) ) ) );
createUser$ = createEffect(() => this.#actions$.pipe(
ofType(UsersActions.createUserRequested), concatMap(({ user }) =>
this.#usersApi.createUser(user).pipe( map(created => UsersActions.createUserSuccess({ user:
created })), catchError(error => of(UsersActions.createUserFailure({ error: error.message
})))  ) ) ) ); login$ = createEffect(() => this.#actions$.pipe(
ofType(UsersActions.loginSubmitted), exhaustMap(({ credentials }) =>
this.#usersApi.login(credentials).pipe( map(user => UsersActions.loginSuccess({ user })),
catchError(error => of(UsersActions.loginFailure({ error: error.message })))  ) ) ) );
navigateAfterLogin$ = createEffect( () => this.#actions$.pipe(
ofType(UsersActions.loginSuccess), tap(() => this.#router.navigate(['/dashboard'])) ), {
dispatch: false } ); saveDocument$ = createEffect(() => this.#actions$.pipe(
ofType(UsersActions.saveDocumentRequested),
withLatestFrom(this.#store.select(selectCurrentDocument)), switchMap(([_action, currentDoc])
=> this.#usersApi.saveDocument(currentDoc).pipe( map(() =>
UsersActions.saveDocumentSuccess()), catchError(error =>
of(UsersActions.saveDocumentFailure({ error: error.message })))  ) ) ) ); }
```

```
// users/users.routes.ts import { Routes } from '@angular/router'; import { provideState }
from '@ngrx/store'; import { provideEffects } from '@ngrx/effects'; import { usersReducer }
from './store/users.reducer'; import { UsersEffects } from './store/users.effects'; import {
UsersPageComponent } from './users-page.component'; export const usersRoutes: Routes = [{
path: '', providers: [ provideState('users', usersReducer), provideEffects([UsersEffects]),
], component: UsersPageComponent, }];
```

Section 4 — Before & After Refactor

Before — HTTP calls inside a component

```
// WRONG: Async logic in the component @Component({}) export class UsersPageComponent
implements OnInit { readonly #http = inject(HttpClient); readonly #store = inject(Store);
ngOnInit(): void { this.#http.get<User[]>('/api/users').subscribe(users => {
this.#store.dispatch(UsersActions.loadUsersSuccess({ users })); // No error handling, no
loading state, no cleanup }); } }
```

What breaks: No loading state management, no error handling, no subscription cleanup, no DevTools visibility into the async operation, no reusability.

After — HTTP call in an Effect

```
// CORRECT: Component is purely declarative @Component({}) export class UsersPageComponent
implements OnInit { readonly #store = inject(Store); loading$ =
this.#store.select(selectUsersLoading); users$ = this.#store.select(selectAllUsers); error$ =
this.#store.select(selectUsersError); ngOnInit(): void {
this.#store.dispatch(UsersActions.loadUsersPageOpened()); } } // Effect handles: HTTP call,
loading state, error handling, DevTools visibility
```

Why: The component becomes a pure view layer — it dispatches events and renders state. All async complexity lives in the Effect where it is independently testable, visible in DevTools, and reusable across any component that dispatches the same action.

Section 5 — Common Mistakes

Mistake 1 — Placing catchError outside the inner Observable

```
// WRONG: catchError on outer stream — one error kills Effect permanently loadUsers$ =
createEffect(() => this.#actions$.pipe( ofType(UsersActions.loadUsersPageOpened),
switchMap(() => this.#usersApi.getUsers().pipe( map(users => UsersActions.loadUsersSuccess({
users })) )), catchError(error => of(UsersActions.loadUsersFailure({ error: error.message
})))) ) ); // CORRECT: catchError inside the inner Observable loadUsers$ = createEffect(() =>
this.#actions$.pipe( ofType(UsersActions.loadUsersPageOpened), switchMap(() =>
this.#usersApi.getUsers().pipe( map(users => UsersActions.loadUsersSuccess({ users })),
catchError(error => of(UsersActions.loadUsersFailure({ error: error.message })))) ) ) ) );
```

What it is: Placing catchError after the flattening operator instead of inside the inner Observable.

Why it happens: It looks cleaner — one catchError at the end reads more linearly.

How to avoid it: Always place catchError inside the switchMap/concatMap callback, chained after the HTTP call.

Angular consequence: The first HTTP error permanently kills the Effect. No error thrown. Users must refresh the page to recover.

Mistake 2 — Using switchMap for write operations

```
// WRONG: switchMap for a DELETE deleteUser$ = createEffect(() => this.#actions$.pipe(
ofType(UsersActions.deleteUserRequested), switchMap(({ userId }) => // WRONG — cancels
previous DELETE mid-flight this.#usersApi.deleteUser(userId).pipe(...) ) ) );
```

What it is: Using switchMap for POST/PUT/PATCH/DELETE operations.

How to avoid it: For any write operation, use concatMap (preserve order) or exhaustMap (ignore duplicates). Reserve switchMap exclusively for reads where 'latest wins' is correct.

Angular consequence: A user double-clicks Delete. Two actions dispatch. switchMap cancels the first DELETE mid-flight. Data integrity is corrupted silently.

Mistake 3 — Forgetting { dispatch: false } on non-dispatching Effects

```
// WRONG: missing { dispatch: false } navigateAfterLogin$ = createEffect(() =>
this.#actions$.pipe( ofType(UsersActions.loginSuccess), tap(() =>
this.#router.navigate(['/dashboard'])) // tap returns original action — NgRx dispatches
loginSuccess again -> infinite loop ) // Missing: { dispatch: false } );
```

What it is: Omitting { dispatch: false } from an Effect that ends with tap() instead of map().

Angular consequence: NgRx dispatches the original action back into the store, triggering the Effect again. Infinite loop floods DevTools with hundreds of identical actions per second.

Section 6 — Do's and Don'ts

DO	DON'T
Place catchError inside the inner Observable (inside switchMap/concatMap)	Place catchError at the outer stream level — one error kills the Effect forever
Use concatMap for write operations (POST/PUT/DELETE)	Use switchMap for writes — silently cancels in-flight mutation requests
Use exhaustMap for login and form submissions	Use mergeMap for login — allows unlimited concurrent login requests
Add { dispatch: false } to every Effect ending with tap()	Omit { dispatch: false } on side-effect-only Effects — creates infinite dispatch loops
Register Effects lazily with provideEffects() in route providers	Register all Effects eagerly in app.config.ts — they run even when the feature isn't loaded
Read store state in Effects with withLatestFrom(store.select(...))	Nest a subscribe() inside an Effect to read current state

Section 7 — Interview Prep

Question 1

"Where should you place catchError in an NgRx Effect and why does placement matter?"

Strong answer covers:

- Must be inside the inner Observable (inside the switchMap/concatMap callback), not on the outer stream
- Outer placement: one HTTP error completes the outer Observable permanently — the Effect dies silently for the rest of the session
- Inner placement: error is caught per-request, outer stream continues, Effect remains alive for future actions
- Fix returns of(failureAction(...)) — of() creates a completing Observable so the inner stream ends cleanly and the outer stream continues listening

Weak answers miss: Why the outer stream dying is silent — no console error, no thrown exception, the app just stops responding to that action type.

Level: Mid

Question 2

"When would you use exhaustMap vs concatMap vs switchMap in an NgRx Effect?"

Strong answer covers:

- switchMap: reads where latest-wins (search, page load on navigation) — cancels previous
- concatMap: writes that must preserve order (sequential create/update/delete)
- exhaustMap: operations where duplicates must be ignored while one is in flight (login, form submit, payment)
- mergeMap: independent parallel operations that all must complete (file uploads, batch operations)
- Cardinal rule: never switchMap for writes — it silently cancels in-flight mutations

Weak answers miss: The specific production consequence of switchMap on a DELETE — the request may be partially processed server-side when cancelled, leaving data in an undefined state.

Level: Mid-Senior

Section 8 — Quick Reference Card

3-line definition: Effects are NgRx's dedicated async layer — they listen to the action stream, perform side effects (HTTP, navigation, analytics) outside the pure reducer, and dispatch new actions back into the store to complete the cycle. The operator after ofType() decides whether in-flight work is cancelled, queued, or ignored on duplicate actions. A misplaced catchError silently kills an Effect permanently on its first error.

Syntax at a glance:

```
// Basic Effect loadUsers$ = createEffect(() => this.#actions$.pipe(
ofType(UsersActions.loadUsersPageOpened), switchMap(() => this.#usersApi.getUsers().pipe(
map(users => UsersActions.loadUsersSuccess({ users })), catchError(err =>
of(UsersActions.loadUsersFailure({ error: err.message })))  ) ) ); // Fire-and-forget (no
dispatch) navigate$ = createEffect( () => this.#actions$.pipe(
ofType(UsersActions.loginSuccess), tap(() => this.#router.navigate(['/dashboard'])) ), {
dispatch: false } ); // Reading state in an Effect save$ = createEffect(() =>
this.#actions$.pipe( ofType(UsersActions.saveRequested),
withLatestFrom(this.#store.select(selectCurrentDoc)), switchMap(([_, doc]) =>
this.#api.save(doc).pipe( map(() => UsersActions.saveSuccess()), catchError(err =>
of(UsersActions.saveFailure({ error: err.message })))  ) ); // Register lazily
provideEffects([UsersEffects])
```

Operator decision:

Scenario	Operator
Load data on page open	switchMap
Search / autocomplete	switchMap
Create / update / delete	concatMap
Login / form submit	exhaustMap
Independent parallel uploads	mergeMap

When to use: Any async operation that must live outside a reducer — HTTP calls, navigation, analytics, localStorage writes, WebSocket subscriptions.

When not to use: Synchronous state transforms — those belong in reducers.

The one rule that matters most: catchError must be inside the switchMap/concatMap inner Observable — one misplaced catchError on the outer stream silently kills the entire Effect after the first error, with no console warning and no way to recover without a page refresh.

TOPIC 53 — NgRx Selectors In Depth

Consolidated | Difficulty: Intermediate-Advanced | Relevance: Critical

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate-Advanced — The basic createSelector API is simple; the difficulty is understanding why memoisation works the way it does, what breaks it, how to compose selectors correctly at scale, and how poorly written selectors silently destroy the performance advantages NgRx is supposed to provide.

Angular Relevance: Critical — Selectors are the most performance-critical piece of NgRx; a single incorrectly written selector can cause every OnPush component in your app to re-render on every action dispatch, completely negating the change detection optimisations you paired NgRx with in the first place.

Section 1 — Explanation

The Spreadsheet Formula Analogy

Imagine a large Excel spreadsheet representing your application state. Raw data lives in cells (the store slices). You don't show users the raw cells — you show them computed formula cells that derive meaningful values: totals, filtered lists, formatted strings.

Excel is smart about recalculation. If cell A1 changes, only formulas that depend on A1 recalculate. A formula depending only on B1 and C1 doesn't rerun just because A1 changed. This is memoisation — recalculate only when the inputs you depend on actually change.

NgRx selectors are those formula cells. createSelector(inputA, inputB, projector) tells NgRx: 'this selector depends on inputA and inputB — only rerun the projector function when inputA or inputB produces a new value.' If an unrelated slice of state changes, the user list selector's projector never runs. The component never re-renders. This is the performance contract selectors make — and it only holds if you follow the rules.

Technical Explanation

createSelector uses referential equality (===) to check whether its input selectors produced new values. If all inputs return the same references as the previous call, the cached result is returned immediately without running the projector.

This works because NgRx reducers (Topic 51) always return new object references on change — a new reference means new data, the same reference means nothing changed. The selector chain tracks this all the way from the raw state slice to the final derived value.

The memoisation chain:

```
Store emits new state -> Feature selector checks: is state['users'] a new reference? -> If NO: return cached result, stop here -> If YES: run next selector -> selectAllUsers checks: is users array a new reference? -> If NO: return cached result -> If YES: run projector -> derive filtered list -> cache result -> emit
```

Every level in this chain can short-circuit. This is why composing selectors from smaller selectors is better than writing one large selector — more opportunities for early short-circuit.

API	Purpose	Key behaviour
createFeatureSelector<T>(key)	Get top-level feature slice	Entry point for all feature selectors
createSelector(inputs, projector)	Derive data from inputs	Memoised — projector only runs on new input references
createSelectorFactory(memoize)	Custom memoisation strategy	Used when default referential equality isn't appropriate
store.select(selector)	Subscribe to a selector	Returns Observable — emits only when selector result changes
store.selectSignal(selector)	Subscribe as a Signal	NgRx 17+ — no async pipe needed, integrates with Signals
selector.projector(args)	Call projector directly	Used in unit tests to test derivation logic in isolation

Section 2 — Connection to Prior Concepts

Directly builds on:

- Topic 51 — NgRx Core Concepts: Basic createSelector and createFeatureSelector were introduced there. This topic goes significantly deeper — composition, memoisation mechanics, factory selectors, and performance pitfalls.

- Topic 24 — OnPush Change Detection: Selectors and OnPush are designed as a pair. OnPush components only re-render when their input references change or an Observable they subscribe to emits. Memoised selectors emit only when their derived value changes — making OnPush effectively free when selectors are written correctly.

- Topic 22 — Angular Signals: NgRx 17+ introduced selectSignal() — store.selectSignal(selector) returns a Signal instead of an Observable, integrating NgRx state directly into the Signals reactivity system without needing the async pipe.

- Topic 14 — readonly & Immutability: Selectors depend on referential equality — which depends on reducers returning new references on change. If a reducer mutates state in place, the selector's input reference doesn't change, the projector never runs, and the component freezes. Immutability is the prerequisite for memoisation to work.

- Topic 52 — NgRx Effects: Effects use withLatestFrom(store.select(selector)) to read current state — understanding selector composition determines how efficiently Effects access the state they need.

Bridging note: In Topic 24 you learned that OnPush only checks a component when its Observable input emits a new value. In Topic 51 you learned that selectors return Observables. The connection: a correctly memoised selector emits only when its specific derived data changes, not on every store emission — meaning an OnPush component re-renders exactly as

often as its data actually changes, and never more.

Most directly extends: Topic 51 — NgRx Core Concepts.

Section 3 — Code Example

Plain TS — Memoisation mechanics in isolation

```
function createSelectorManual<S, R1, Result>( input1: (state: S) => R1, projector: (r1: R1) => Result ) { let lastInput1: R1 | undefined; let lastResult: Result | undefined; return (state: S): Result => { const newInput1 = input1(state); if (newInput1 === lastInput1 && lastResult !== undefined) { return lastResult; // Cache hit } lastResult = projector(newInput1); // Cache miss lastInput1 = newInput1; return lastResult; }; } const state = { users: [{ id: 1, name: 'Alice' }], loading: false }; const selectUsers = (s: typeof state) => s.users; const selectUserCount = createSelectorManual(selectUsers, users => users.length); selectUserCount(state); // Cache miss -> 1 selectUserCount(state); // Cache hit -> 1 const state2 = { ...state, loading: true }; // new object, same users reference selectUserCount(state2); // Cache hit — users reference unchanged, projector skipped
```

Real Angular — Full selector composition hierarchy

```
// users/store/users.selectors.ts import { createFeatureSelector, createSelector, createSelectorFactory, resultMemoize } from '@ngrx/store'; import { isEqual } from 'lodash'; import { UsersState } from './users.reducer'; // Layer 1: Feature selector const selectUsersState = createFeatureSelector<UsersState>('users'); // Layer 2: Slice selectors — one per state property export const selectAllUsers = createSelector(selectUsersState, state => state.users); export const selectUsersLoading = createSelector(selectUsersState, state => state.loading); export const selectUsersError = createSelector(selectUsersState, state => state.error); export const selectSearchQuery = createSelector(selectUsersState, state => state.searchQuery); export const selectSelectedUserId = createSelector(selectUsersState, state => state.selectedUserId); // Layer 3: Derived selectors export const selectFilteredUsers = createSelector( selectAllUsers, selectSearchQuery, (users, query) => { if (!query.trim()) return users; const lower = query.toLowerCase(); return users.filter(u => u.name.toLowerCase().includes(lower) || u.email.toLowerCase().includes(lower) ); } ); export const selectFilteredUserCount = createSelector(selectFilteredUsers, users => users.length); // Layer 4: View model selector — one per component export const selectUsersPageViewModel = createSelector( selectFilteredUsers, selectUsersLoading, selectUsersError, selectFilteredUserCount, selectSearchQuery, (users, loading, error, count, query) => ({ users, loading, error, count, query, hasResults: users.length > 0, isEmpty: !loading && users.length === 0 && !error }) ); // Parameterised selector — factory function pattern export const selectUserById = (id: number) => createSelector(selectAllUsers, users => users.find(u => u.id === id) ?? null); // Custom memoisation for unavoidable new references const createDeepEqualSelector = createSelectorFactory(projector => resultMemoize(projector, isEqual) ); export const selectSortedUserIds = createDeepEqualSelector( selectAllUsers, users => [...users].sort((a, b) => a.id - b.id).map(u => u.id) );
```

Component — Observable pattern with view model selector

```
@Component({ selector: 'app-users-page', standalone: true, imports: [AsyncPipe], changeDetection: ChangeDetectionStrategy.OnPush, template: ` @if (vm$ | async; as vm) { <input [value]="vm.query" (input)="onSearch($event)" /> <span>{{ vm.count }} results</span> @if (vm.loading) { <app-spinner /> } @if (vm.isEmpty) { <p>No users found for "{{ vm.query }}"</p> } @for (user of vm.users; track user.id) { <app-user-card [user]="user" /> } } ` }) export class UsersPageComponent implements OnInit { readonly #store = inject(Store); vm$ = this.#store.select(selectUsersPageViewModel); // ONE subscription ngOnInit(): void { this.#store.dispatch(UsersActions.loadUsersPageOpened()); } onSearch(event: Event): void { const query = (event.target as HTMLInputElement).value; this.#store.dispatch(UsersActions.userSearched({ query })); } }
```

Component — Signal pattern (NgRx 17+)

```
@Component({ standalone: true, changeDetection: ChangeDetectionStrategy.OnPush, template: `
@if (vm().loading) { <app-spinner /> } @for (user of vm().users; track user.id) {
<app-user-card [user]="user" /> } ` }) export class UsersPageSignalComponent { readonly
#store = inject(Store); vm = this.#store.selectSignal(selectUsersPageViewModel); // Signal —
no async pipe }
```

Parameterised selector — modern input() pattern

```
@Component({ selector: 'app-user-card', standalone: true, imports: [AsyncPipe],
changeDetection: ChangeDetectionStrategy.OnPush, template: `@if (user$ | async; as user) {
<h3>{{ user.name }}</h3> }` }) export class UserCardComponent implements OnInit { readonly
userId = input.required<number>(); // Signal-based input readonly #store = inject(Store);
user$!: Observable<User | null>; ngOnInit(): void { this.user$ =
this.#store.select(selectUserById(this.userId())); } }
```

Section 4 — Before & After Refactor

Before — Inline selectors and template logic

```
// WRONG @Component({ template: ` <span>{{ getFilteredCount() }} users</span> <!-- reruns
every CD cycle --> @for (user of (users$ | async)?.filter(u => u.name.includes(searchQuery));
track user.id) { ... } ` }) export class UsersComponent { readonly #store = inject(Store); //
Inline lambda — NgRx cannot memoise anonymous function users$ = this.#store.select(state =>
state['users']['users']); loading$ = this.#store.select(state => state['users']['loading']);
searchQuery = ''; getFilteredCount(): number { return this.users.filter(u =>
u.name.includes(this.searchQuery)).length; } }
```

What breaks: Filtering runs on every change detection cycle. Inline lambdas in store.select() are new function references every render — NgRx cannot memoise them. Multiple separate subscriptions mean multiple potential emissions per state update.

After — Composed view model selector

```
// CORRECT @Component({ changeDetection: ChangeDetectionStrategy.OnPush, template: ` @if (vm$
| async; as vm) { <span>{{ vm.count }} users</span> @for (user of vm.users; track user.id) {
<app-user-card [user]="user" /> } } ` }) export class UsersComponent { readonly #store =
inject(Store); vm$ = this.#store.select(selectUsersPageViewModel); // ONE subscription }
```

Why: The filtering and counting projectors only run when the users array or search query actually changes. With OnPush, the component re-renders only when vm$ emits a new object — which only happens when the memoised result changes.

Section 5 — Common Mistakes

Mistake 1 — Using inline lambda functions in store.select()

```
// WRONG: Inline lambda — new function reference every render users$ =
this.#store.select(state => state['users'].users); filteredUsers$ = this.#store.select(state
=> state['users'].users.filter(u => u.active)); // CORRECT: Named selector, stable reference,
NgRx can memoise users$ = this.#store.select(selectAllUsers); filteredUsers$ =
this.#store.select(selectActiveUsers);
```

Angular consequence: The filter computation runs on every single action dispatched anywhere in the app. With 10 components doing this, every action triggers 10 filter computations simultaneously.

Mistake 2 — Returning new object or array literals from projectors

```
// WRONG: New object reference every call — breaks memoisation export const selectUserSummary
= createSelector(selectAllUsers, users => ({ count: users.length, hasAdmins: users.some(u =>
u.role === 'admin') }) // ^ NEW object literal on every call ); // CORRECT: Return primitives
where possible export const selectUserCount = createSelector(selectAllUsers, users =>
users.length); export const selectHasAdmins = createSelector(selectAllUsers, users =>
users.some(u => u.role === 'admin'));
```

Angular consequence: Every OnPush component subscribed to selectUserSummary re-renders on every action that touches users state, even when count and hasAdmins are unchanged. OnPush is completely defeated.

Mistake 3 — Recreating parameterised selectors inside component methods

```
// WRONG: New selector instance on every CD cycle @Component({ template: `@if
(store.select(getUserSelector()) | async; as user) { {{ user.name }} }` }) export class
UserCardComponent { getUserSelector() { return createSelector(selectAllUsers, users =>
users.find(u => u.id === this.userId())); // new every call } } // CORRECT: Create once in
ngOnInit — stable reference export class UserCardComponent implements OnInit { readonly
userId = input.required<number>(); readonly #store = inject(Store); user$!: Observable<User |
null>; ngOnInit(): void { this.user$ = this.#store.select(selectUserById(this.userId())); } }
```

Angular consequence: A new selector instance with empty cache is created every cycle. For 100 user cards, 100 filter operations run on every action dispatch.

Section 6 — Do's and Don'ts

DO	DON'T
Compose selectors in layers: feature -> slice -> derived -> view model	Write one large selector that does all derivation in a single projector
Return primitive values from projectors when possible	Return new {} or [] literals from projectors — they break referential equality
Create parameterised selectors once in ngOnInit or with effect()	Call createSelector() inside template-bound methods — new instance every CD cycle
Use store.selectSignal(selector) in Signal-based components	Use inline lambdas in store.select(state => ...) — NgRx cannot memoise them
Write one view model selector per component	Subscribe to five separate selectors and combine them in the template
Test projectors directly with selector.projector(mockInput)	Set up a full store just to test projector logic — it's a pure function

Section 7 — Interview Prep

Question 1

"How does NgRx selector memoisation work, and what can break it?"

Strong answer covers:

- Selectors use referential equality (===) to compare input values between calls

- If all inputs return the same references, the projector is skipped and the cached result is returned

- This works because reducers always return new references on change

- Three things that break it: inline lambdas in store.select() (new function reference each time), object/array literals in projectors (always new reference), and mutations in reducers (same reference despite changed data)

- Fix for unavoidable new references: createSelectorFactory with resultMemoize and deep equality

Weak answers miss: The connection to reducer immutability — memoisation depends entirely on reducers returning new references, which depends on never mutating state.

Level: Mid-Senior

Question 2

"What is a view model selector and why use one instead of multiple separate store.select() calls?"

Strong answer covers:

- A view model selector combines all state a specific component needs into one derived object

- Benefits: one Observable subscription instead of many, one emission per state change, all derivation logic in a testable pure function outside the component

- The component template becomes a pure rendering layer with zero logic

- Trade-off: the view model object is a new reference on every emission — input selectors must return primitives or stable references to control emission frequency

- store.selectSignal(viewModelSelector) integrates directly with Signal-based templates — no async pipe needed

Weak answers miss: The emission frequency trade-off — a view model returning a new object on every action dispatch defeats OnPush if the input selectors aren't correctly composed.

Level: Mid

Section 8 — Quick Reference Card

3-line definition: NgRx selectors are memoised pure functions that short-circuit recomputation when their input references haven't changed — they are the performance engine of NgRx, and every decision about how you write them directly determines whether OnPush change detection actually works as intended. Compose them in layers: feature -> slice -> derived -> view model. The more layers, the more short-circuit opportunities.

Syntax at a glance:

```
// Layer 1 — feature const selectUsersState = createFeatureSelector<UsersState>('users'); // Layer 2 — raw slice export const selectAllUsers = createSelector(selectUsersState, s => s.users); export const selectSearchQuery = createSelector(selectUsersState, s => s.searchQuery); // Layer 3 — derived export const selectFilteredUsers = createSelector( selectAllUsers, selectSearchQuery, (users, query) => users.filter(u => u.name.includes(query)) ); // Layer 4 — view model export const selectUsersPageVm = createSelector( selectFilteredUsers, selectUsersLoading, selectUsersError, (users, loading, error) => ({ users, loading, error, count: users.length }) ); // Component — Observable vm$ = this.#store.select(selectUsersPageVm); // Component — Signal (NgRx 17+) vm = this.#store.selectSignal(selectUsersPageVm); // Parameterised — factory export const selectUserById = (id: number) => createSelector(selectAllUsers, users => users.find(u => u.id === id) ?? null); // Custom memoisation const createDeepEqualSelector = createSelectorFactory(p => resultMemoize(p, isEqual));
```

When to use: Every time a component needs data from the store — always via a named selector, never via inline lambda.

When not to use: There is no case where skipping selectors is correct. Even for a single primitive value, a named slice selector is the right pattern.

The one rule that matters most: Returning a new object or array literal from a projector breaks memoisation — the selector emits on every action that touches its inputs, OnPush stops working, and the app renders as frequently as with default change detection — silently, with no warning.

TOPIC 54 — NgRx Entity

Consolidated | Difficulty: Intermediate | Relevance: High

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — NgRx Entity introduces a new abstraction layer (the EntityAdapter) on top of the NgRx reducer pattern you already know, requiring you to learn a new API rather than new programming concepts.

Angular Relevance: High — Any Angular app managing lists of records (users, products, orders, messages) benefits directly from NgRx Entity, which eliminates hundreds of lines of hand-written CRUD reducer boilerplate while enforcing a consistent, performant data structure.

Section 1 — Explanation

The Filing Cabinet Analogy

Imagine you're managing paper records for a company. Without a system, you pile every employee file in a heap — to find one, you rifle through the whole stack. To update one record, you pull out the stack, find it, swap it, and put everything back. Slow and error-prone.

A professional filing cabinet solves this: every file gets a unique tab ID, files are stored alphabetically by ID, and you have a master index listing all IDs in order. To find, update, or delete any record, you go straight to the tab. Done.

NgRx Entity is that filing cabinet for your NgRx store.

Technically:

NgRx Entity (@ngrx/entity) provides an EntityAdapter that manages a normalised data structure called an EntityState:

```
interface EntityState<T> { ids: string[] | number[]; // ordered list of all entity IDs
entities: Record<string, T>; // dictionary: id -> entity object }
```

Instead of writing your own find, filter, map, and splice logic in reducers, the adapter gives you ready-made CRUD operations that return new state objects, compatible with NgRx's immutability requirements.

Method	What it does
addOne(entity, state)	Adds one entity; ignores if ID exists
setOne(entity, state)	Adds or replaces one entity
setAll(entities, state)	Replaces entire collection
updateOne({ id, changes }, state)	Partial update on one entity
upsertOne(entity, state)	Add if missing, merge if exists
removeOne(id, state)	Removes one by ID

Method	What it does
getInitialState(extra?)	Creates { ids: [], entities: {} }

Section 2 — Connection to Prior Concepts

Directly builds on:

- Topic 51 — NgRx Core Concepts: The reducer pattern, pure state functions, createReducer, on(), and createFeatureSelector/createSelector are all prerequisites. NgRx Entity simply provides helper functions that slot inside reducers you already know how to write.

- Topic 53 — NgRx Selectors In Depth: The adapter generates built-in selectors (selectAll, selectEntities, selectIds, selectTotal) that plug directly into the composed selector pattern from Topic 53.

- Topic 52 — NgRx Effects: Entity reducers are fed by actions dispatched from Effects — the HTTP-to-store pattern from Topic 52 is where entity data typically enters the store.

- Topic 14 — readonly & Immutability: The adapter's CRUD methods always return new state references, enforcing the immutability rule from Topic 14 automatically.

Bridging note: In Topic 51 you wrote reducers like state.users.filter(u => u.id !== action.id) for every CRUD operation. NgRx Entity replaces every one of those array operations with a single adapter method call, while the normalised { ids, entities } structure makes selector lookups O(1) instead of O(n) array scans.

Most directly extends: Topic 51 — NgRx Core Concepts.

Section 3 — Code Example

Plain TS — What EntityAdapter does under the hood

```
// Without Entity — hand-written, error-prone function removeUser(state: UsersState, id:
string): UsersState { return { ...state, users: state.users.filter(u => u.id !== id) }; //
O(n) scan } // With Entity: adapter.removeOne(id, state) — O(1) dictionary delete
```

Full Angular NgRx Entity Feature

```
// user.model.ts export interface User { id: string; name: string; email: string; role:
'admin' | 'viewer'; } // user.actions.ts import { createActionGroup, emptyProps, props } from
'@ngrx/store'; export const UsersApiActions = createActionGroup({ source: 'Users API',
events: { 'Users Loaded': props<{ users: User[] }>(), 'User Added': props<{ user: User }>(),
'User Updated': props<{ id: string; changes: Partial<User> }>(), 'User Removed': props<{ id:
string }>(), 'Users Load Failed': props<{ error: string }>(), }}); export const
UsersPageActions = createActionGroup({ source: 'Users Page', events: { 'Page Opened':
emptyProps(), 'Remove User Clicked': props<{ id: string }>(), }});
```

```
// user.reducer.ts import { createEntityAdapter, EntityAdapter, EntityState } from '@ngrx/entity'; import { createFeature, createReducer, on } from '@ngrx/store'; export const userAdapter: EntityAdapter<User> = createEntityAdapter<User>({ sortComparer: (a, b) => a.name.localeCompare(b.name), // Sort maintained at write time }); export interface UsersState extends EntityState<User> { loading: boolean; error: string | null; selectedUserId: string | null; } const initialState: UsersState = userAdapter.getInitialState({ loading: false, error: null, selectedUserId: null, }); export const usersFeature = createFeature({ name: 'users', reducer: createReducer(initialState, on(UsersApiActions.usersLoaded, (state, { users }) => userAdapter.setAll(users, { ...state, loading: false, error: null }) ), on(UsersApiActions.userAdded, (state, { user }) => userAdapter.addOne(user, state)), on(UsersApiActions.userUpdated, (state, { id, changes }) => userAdapter.updateOne({ id, changes }, state)), on(UsersApiActions.userRemoved, (state, { id }) => userAdapter.removeOne(id, state)), on(UsersApiActions.usersLoadFailed, (state, { error }) => ({ ...state, loading: false, error })), ), }); export const { selectUsersState, selectLoading, selectError, selectSelectedUserId } = usersFeature;
```

```
// user.selectors.ts import { createSelector } from '@ngrx/store'; import { userAdapter, selectUsersState, selectSelectedUserId } from './user.reducer'; // ALWAYS pass the feature selector to getSelectors() const { selectAll: selectAllUsers, selectEntities: selectUserEntities, selectTotal: selectUserTotal } = userAdapter.getSelectors(selectUsersState); export { selectAllUsers, selectUserEntities, selectUserTotal }; export const selectSelectedUser = createSelector( selectUserEntities, selectSelectedUserId, (entities, selectedId) => selectedId ? entities[selectedId] ?? null : null // O(1) dictionary lookup — no array find() );
```

```
// users.component.ts @Component({ selector: 'app-users', standalone: true, changeDetection: ChangeDetectionStrategy.OnPush, template: ` @if (loading()) { <p>Loading...</p> } @for (user of users(); track user.id) { <div>{{ user.name }}<button (click)="remove(user.id)">Remove</button></div> } <p>Total: {{ total() }}</p> ` }) export class UsersComponent implements OnInit { readonly #store = inject(Store); users = this.#store.selectSignal(selectAllUsers); loading = this.#store.selectSignal(selectLoading); total = this.#store.selectSignal(selectUserTotal); ngOnInit(): void { this.#store.dispatch(UsersPageActions.pageOpened()); } remove(id: string): void { this.#store.dispatch(UsersPageActions.removeUserClicked({ id })); } }
```

Section 4 — Before & After Refactor

Before — Hand-written array CRUD reducer

```
// WRONG: Writing all array operations manually on(UsersApiActions.usersLoaded, (state, { users }) => ({ ...state, users, loading: false })), on(UsersApiActions.userUpdated, (state, { id, changes }) => ({ ...state, users: state.users.map(u => u.id === id ? { ...u, ...changes } : u), // O(n) scan })), on(UsersApiActions.userRemoved, (state, { id }) => ({ ...state, users: state.users.filter(u => u.id !== id), // O(n) scan })),
```

What breaks: O(n) operations on every state change. Selector by ID requires users.find() — another O(n) scan. Duplicate IDs not prevented. Sort order must be reimplemented manually on every operation.

After — NgRx Entity adapter

```
// CORRECT: adapter handles all CRUD operations on(UsersApiActions.usersLoaded, (state, { users }) => userAdapter.setAll(users, { ...state, loading: false }) // replaces all ), on(UsersApiActions.userUpdated, (state, { id, changes }) => userAdapter.updateOne({ id, changes }, state) // O(1) ), on(UsersApiActions.userRemoved, (state, { id }) => userAdapter.removeOne(id, state) // O(1) ),
```

Why: Selector performance is O(1) via dictionary lookup. Duplicate protection is built in. Sort order maintained automatically by sortComparer. Adapter methods are already tested by the

NgRx team.

Section 5 — Common Mistakes

Mistake 1 — Using addOne when you need upsertOne for API sync

```
// WRONG: addOne silently ignores if entity already exists on(UsersApiActions.usersRefreshed,
(state, { users }) => users.reduce((s, user) => userAdapter.addOne(user, s), state) // stale
data shown silently // CORRECT: upsertMany adds if new, merges if exists
on(UsersApiActions.usersRefreshed, (state, { users }) => userAdapter.upsertMany(users,
state))
```

Angular consequence: UI silently shows stale data because the old entity was never replaced. No error, no warning.

Mistake 2 — Calling getSelectors() without passing the feature selector

```
// WRONG — reads from root state, not the feature slice const { selectAll } =
userAdapter.getSelectors(); // CORRECT — always scope to the feature const { selectAll } =
userAdapter.getSelectors(selectUsersState);
```

Angular consequence: selectAll returns undefined or throws Cannot read properties of undefined because it reads ids and entities from the wrong state level.

Mistake 3 — Sorting inside a selector instead of using sortComparer

```
// WRONG: Sorting in selector — new array reference on every emission export const
selectSortedUsers = createSelector( selectAllUsers, users => [...users].sort((a, b) =>
a.name.localeCompare(b.name)) // breaks memoisation ); // CORRECT: Define sort in the adapter
— maintained at write time export const userAdapter = createEntityAdapter<User>({
sortComparer: (a, b) => a.name.localeCompare(b.name), }); // selectAll already returns users
pre-sorted — no extra computation
```

Angular consequence: Unnecessary OnPush re-renders on every action touching the users slice. Topic 53 memoisation rule violated silently.

Section 6 — Do's and Don'ts

DO	DON'T
Define sortComparer in createEntityAdapter() for ordered lists	Sort inside selectors — returns new array reference every emission
Use upsertOne/upsertMany when syncing from an API	Use addOne for API refreshes — silently ignores existing records
Always pass the feature selector to getSelectors(featureSelector)	Call getSelectors() with no argument — produces root-scoped selectors
Use updateOne({ id, changes }) for partial updates on existing records	Spread the whole entity manually in the reducer
Extend EntityState<T> with extra slices (loading, error, selectedId)	Force loading/error flags into entity objects themselves

DO	DON'T
Register provideState('users', usersReducer) lazily in route providers	Register entity feature state in app.config.ts — eager loads all entities

Section 7 — Interview Prep

Question 1

"What problem does NgRx Entity solve, and how does its data structure improve performance over a plain array in the store?"

Strong answer covers:

- Arrays require O(n) scan for find/update/delete; Entity normalises to { ids, entities } enabling O(1) dictionary lookup
- Eliminates hand-written CRUD boilerplate in reducers (map, filter, spread) with battle-tested adapter methods
- The ids array preserves ordered selection while the entities dictionary enables fast keyed access
- Adapter methods automatically return new state references — satisfying NgRx immutability requirements
- sortComparer maintains order automatically without repeated sorting in selectors

Weak answers miss: The performance argument (O(1) vs O(n)) — most candidates explain the API without explaining why the normalised structure matters.

Level: Mid

Question 2

"What is the difference between addOne, setOne, and upsertOne, and when would you use each?"

Strong answer covers:

- addOne: adds the entity only if the ID is absent — silently ignores if already present
- setOne: unconditionally adds or replaces the entire entity
- upsertOne: adds if absent, merges (shallow) if present — like a database UPSERT
- addOne for user-created records (guaranteed new), upsertOne for API sync (may already exist), setOne for guaranteed full replacement
- All three return new state references — they never mutate

Weak answers miss: The distinction between setOne (full replace) and upsertOne (merge) — candidates often treat them as synonyms.

Level: Mid

Section 8 — Quick Reference Card

3-line definition: NgRx Entity normalises a collection of records into a { ids[], entities{} } dictionary structure and provides an EntityAdapter with O(1) CRUD methods that write immutable reducers for you. It eliminates hand-written array operations in reducers while making selector lookups O(1) instead of O(n). Pair it with sortComparer to maintain order at write time — never at read time.

Operation comparison:

Operation	Array (before)	Entity (after)
Lookup by ID	users.find(u => u.id === id) — O(n)	entities[id] — O(1)
Update	users.map(u => u.id === id ? {...u,...c} : u)	updateOne({ id, changes }, state)
Remove	users.filter(u => u.id !== id)	removeOne(id, state)
Prevent duplicates	Manual check required	addOne ignores automatically

When to use: Any NgRx feature managing a list of identifiable records — users, products, orders, messages, threads.

When not to use: Non-collection state (loading flags, form values, UI toggles) — those stay as plain state slices alongside EntityState.

The one rule that matters most: Never sort inside a selector on top of selectAll — define sortComparer in the adapter so order is maintained at write time, not recomputed at read time, and memoisation stays intact.

TOPIC 55 — NgRx Component Store

Consolidated | Difficulty: Intermediate | Relevance: High

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — Component Store introduces a new mental model (local vs global state) and a reactive updater/effect pattern that feels different from the global NgRx store, but builds directly on RxJS and NgRx concepts you already know.

Angular Relevance: High — Component Store solves a real gap in Angular architecture: state that is too complex for a single component's signals but too local and short-lived to justify polluting the global NgRx store — it is the right tool for self-contained feature sections like data tables, wizards, and sidepanels.

Section 1 — Explanation

The Whiteboard Analogy

Imagine your company has two types of information storage: The central database (NgRx global store) — holds company-wide records that every department needs: employee roster, product catalogue, financial data. It persists across the whole app session. Everyone reads from it.

A personal whiteboard in each meeting room — used only during a meeting to track progress on that specific discussion. When the meeting ends, the whiteboard gets wiped. Nobody else in the building cares what's on it. No approval process needed — whoever's in the room can write directly.

NgRx Component Store is the whiteboard. It gives a single component (or a small feature section) its own self-contained reactive state with the same clean patterns as the global store — but scoped to the component's lifetime, requiring no actions, no reducers registered globally, and no DevTools ceremony for ephemeral UI state.

Technically:

@ngrx/component-store is a standalone state management service. You extend ComponentStore<StateType> and get: setState() — replaces or partially updates state; updater() — creates a reusable state mutation function (like a mini-reducer); select() — creates a memoised selector Observable from state; effect() — creates a side-effect handler (like NgRx Effects, but local); patchState() — partial state update; Automatic subscription cleanup when the component is destroyed.

	NgRx Global Store	NgRx Component Store
Scope	Entire application	One component + its children
Lifetime	App session	Component lifetime
State setup	createFeature + provideState()	Extend ComponentStore<T>
Write mechanism	Dispatch actions -> reducers	updater() or patchState()

	NgRx Global Store	NgRx Component Store
Side effects	createEffect() in Effects class	effect() method
DevTools support	Full time-travel debugging	Limited (no action log)
Boilerplate	High	Low

Section 2 — Connection to Prior Concepts

Directly builds on:

- Topic 51 — NgRx Core Concepts: The global store's one-way data flow is mirrored locally in Component Store. You already understand pure state updates and selector memoisation — Component Store uses the same mental model with less ceremony.

- Topic 52 — NgRx Effects: The effect() method in Component Store works identically to NgRx Effects — same switchMap/concatMap/exhaustMap rules apply.

- Topic 26 — RxJS Subscription Management and Topic 27 — takeUntilDestroyed: Component Store handles its own subscription cleanup automatically when the component is destroyed.

- Topic 44 — State Management Patterns: The private state + public read-only projection + public write methods pattern from Topic 44's store service is exactly what Component Store formalises with updater() and select().

- Topic 25 — Services & Dependency Injection: Component Store is provided in the component's providers: [] array — it lives in the component's injector, not the root injector.

Most directly extends: Topic 44 — State Management Patterns.

Section 3 — Code Example

Plain TS — What Component Store does under the hood

```
// Without Component Store — hand-rolled (Topic 44 pattern) @Injectable() class SearchStore {
#state = new BehaviorSubject({ query: '', results: [], loading: false }); readonly state$ =
this.#state.asObservable(); setQuery(query: string): void { this.#state.next({
...this.#state.getValue(), query }); } // Manual cleanup, no RxJS effect management } // With
Component Store — same idea, less boilerplate @Injectable() class SearchStore extends
ComponentStore<SearchState> { readonly setQuery = this.updater((state, query: string) => ({
...state, query })); readonly query$ = this.select(state => state.query); }
```

Full Angular Component Store Feature — Data Table

```
// data-table.store.ts import { Injectable } from '@angular/core'; import { ComponentStore } from '@ngrx/component-store'; import { tapResponse } from '@ngrx/operators'; // from @ngrx/operators in v15+ import { switchMap } from 'rxjs'; import { Observable } from 'rxjs'; import { UserService } from './user.service'; import { User } from './user.model'; interface DataTableState { users: User[]; loading: boolean; error: string | null; filter: string; currentPage: number; pageSize: number; } const initialState: DataTableState = { users: [], loading: false, error: null, filter: '', currentPage: 1, pageSize: 10, }; @Injectable() export class DataTableStore extends ComponentStore<DataTableState> { // Constructor injection required — inject() cannot be used before super() constructor(private readonly userService: UserService) { super(initialState); // Required — initialises internal BehaviorSubject } // SELECTORS readonly users$ = this.select(state => state.users); readonly loading$ = this.select(state => state.loading); readonly error$ = this.select(state => state.error); readonly filter$ = this.select(state => state.filter); readonly currentPage$ = this.select(state => state.currentPage); readonly filteredUsers$ = this.select( this.users$, this.filter$, (users, filter) => filter ? users.filter(u => u.name.toLowerCase().includes(filter.toLowerCase())) : users ); readonly vm$ = this.select( this.filteredUsers$, this.loading$, this.error$, this.currentPage$, (users, loading, error, currentPage) => ({ users, loading, error, currentPage }) ); // UPDATERS readonly setFilter = this.updater((state, filter: string) => ({ ...state, filter, currentPage: 1 })); readonly setPage = this.updater((state, currentPage: number) => ({ ...state, currentPage })); readonly clearError = this.updater(state => ({ ...state, error: null })); // EFFECTS readonly loadUsers = this.effect((trigger$: Observable<void>) => trigger$.pipe( switchMap(() => { this.patchState({ loading: true, error: null }); return this.userService.getUsers().pipe( tapResponse( (users) => this.patchState({ users, loading: false }), (error: Error) => this.patchState({ error: error.message, loading: false }) ) ); }) ) ); }
```

```
// data-table.component.ts @Component({ selector: 'app-data-table', standalone: true, imports: [AsyncPipe], changeDetection: ChangeDetectionStrategy.OnPush, providers: [DataTableStore], // Scoped to this component's lifetime template: ` @if (vm$ | async; as vm) { @if (vm.loading) { <p>Loading...</p> } @if (vm.error) { <p class="error">{{ vm.error }}</p> } <input (input)="onFilterChange($event)" placeholder="Search users..." /> @for (user of vm.users; track user.id) { <div>{{ user.name }}</div> } <button (click)="prevPage(vm.currentPage)">Previous</button> <button (click)="nextPage(vm.currentPage)">Next</button> } ` }) export class DataTableComponent implements OnInit { readonly #store = inject(DataTableStore); vm$ = this.#store.vm$; ngOnInit(): void { this.#store.loadUsers(); } onFilterChange(event: Event): void { this.#store.setFilter((event.target as HTMLInputElement).value); } prevPage(current: number): void { if (current > 1) this.#store.setPage(current - 1); } nextPage(current: number): void { this.#store.setPage(current + 1); } }
```

Section 4 — Before & After Refactor

Before — Complex local state managed inside the component

```
// WRONG: all state crammed into the component class @Component({}) export class DataTableComponent implements OnInit, OnDestroy { users: User[] = []; loading = false; error: string | null = null; filter = ''; currentPage = 1; private destroy$ = new Subject<void>(); constructor(private readonly userService: UserService) {} ngOnInit(): void { this.userService.getUsers().pipe(takeUntil(this.destroy$)).subscribe({ next: users => { this.users = users; this.loading = false; }, error: err => { this.error = err.message; this.loading = false; } }); } get filteredUsers(): User[] { // Recomputes every CD cycle — no memoisation return this.users.filter(u => u.name.toLowerCase().includes(this.filter.toLowerCase())); } ngOnDestroy(): void { this.destroy$.next(); this.destroy$.complete(); } }
```

What breaks: State scattered as individual class properties. filteredUsers getter recomputes on every CD cycle. Business logic in the component — untestable in isolation. Manual Subject + takeUntil boilerplate.

After — Component Store

```
// CORRECT: Component thin, owns all state, selectors memoised, auto-cleaned @Component({
standalone: true, providers: [DataTableStore], changeDetection:
ChangeDetectionStrategy.OnPush, }) export class DataTableComponent implements OnInit {
readonly #store = inject(DataTableStore); vm$ = this.#store.vm$; // Single memoised
subscription ngOnInit(): void { this.#store.loadUsers(); } onFilter(event: Event): void {
this.#store.setFilter((event.target as HTMLInputElement).value); } }
```

Why: The store is independently testable. select() is memoised — filteredUsers$ only recomputes when users$ or filter$ emit. Cleanup is automatic — no Subject, no takeUntil, no ngOnDestroy. Child components can inject the same DataTableStore instance via the component injector hierarchy.

Section 5 — Common Mistakes

Mistake 1 — Providing Component Store in root instead of the component

```
// WRONG — singleton that never resets @Injectable({ providedIn: 'root' }) export class
DataTableStore extends ComponentStore<DataTableState> { } // CORRECT — tied to component
lifetime @Component({ providers: [DataTableStore] }) export class DataTableComponent { }
```

Angular consequence: A root-provided Component Store is a singleton — persists for the entire app session, accumulates state across every component instance, never resets. Opening a data table modal twice shows stale state from the first open.

Mistake 2 — Mutating state directly inside updater()

```
// WRONG — mutation, same reference returned readonly addUser = this.updater((state, user:
User) => { state.users.push(user); return state; // same object reference }); // CORRECT — new
state object with spread readonly addUser = this.updater((state, user: User) => ({ ...state,
users: [...state.users, user] }));
```

Angular consequence: Memoised select() derivations use referential equality — returning the same state object means select() never detects a change, template displays stale data with OnPush.

Mistake 3 — Not using tapResponse and letting errors crash the effect

```
// WRONG — unhandled error kills the effect permanently readonly loadUsers =
this.effect((trigger$: Observable<void>) => trigger$.pipe( switchMap(() =>
this.userService.getUsers().pipe( tap(users => this.patchState({ users })) // No error
handling )) ) ); // CORRECT — tapResponse from @ngrx/operators keeps the effect alive readonly
loadUsers = this.effect((trigger$: Observable<void>) => trigger$.pipe( switchMap(() =>
this.userService.getUsers().pipe( tapResponse( (users) => this.patchState({ users, loading:
false }), (error: Error) => this.patchState({ error: error.message, loading: false }) ) )) )
);
```

Angular consequence: The first HTTP error permanently kills the effect. Calling the store method again silently does nothing — no error, no console warning. The component appears frozen.

Section 6 — Do's and Don'ts

DO	DON'T
Provide Component Store in the component's providers: [] array	Use providedIn: 'root' — creates a persistent singleton that never resets

DO	DON'T
Import tapResponse from @ngrx/operators and use it in every effect()	Use plain tap() with no error handling — permanently kills the effect on first error
Return a new state object from every updater() — { ...state, changed }	Mutate the state argument directly — breaks select() memoisation silently
Use the composed view model selector (vm$) for a single template subscription	Subscribe to each state slice separately — multiple subscriptions, multiple CD triggers
Use patchState() for simple partial updates that don't need payload-based logic	Always use updater() for everything — patchState() is simpler for flag flips
Call super(initialState) as the first line of the constructor	Omit super() — leaves the internal BehaviorSubject uninitialised

Section 7 — Interview Prep

Question 1

"What is NgRx Component Store and how does it differ from the global NgRx Store? When would you choose one over the other?"

Strong answer covers:

- Global Store: app-wide lifetime, accessed anywhere, requires actions/reducers/effects wiring, full DevTools support
- Component Store: component lifetime, scoped to injector subtree, no action ceremony, simpler API
- Choose global store for: data shared across features, data that must survive navigation, data needed by unrelated parts of the app
- Choose Component Store for: UI state local to one feature (pagination, filters, loading), wizard state, per-widget independent state
- The key question: 'Does anything outside this component need this state?' — if no, Component Store

Weak answers miss: The lifetime scoping argument — most candidates explain the API difference but not the lifecycle/ownership distinction.

Level: Mid

Question 2

"What is tapResponse in NgRx Component Store and why should you use it instead of plain tap inside an effect()?"

Strong answer covers:

- tapResponse(successFn, errorFn) from @ngrx/operators catches errors inside the inner Observable

- Plain tap() with no error handling: an HTTP error propagates to the outer effect stream and permanently kills it — same as the Topic 52 catchError placement rule

- After the outer stream errors, calling the effect again silently does nothing — no error, no console warning

- tapResponse ensures errors never reach the outer stream — the effect stays alive for the component's lifetime

Weak answers miss: That the effect is permanently killed (not just skipped) when an unhandled error reaches the outer stream.

Level: Mid

Section 8 — Quick Reference Card

3-line definition: NgRx Component Store is a self-contained, component-scoped reactive state manager — the same updater/selector/effect discipline as NgRx, but with no global registration, no actions, and automatic cleanup when the component is destroyed. Provide it in the component's providers array, call super(initialState) in the constructor, and use tapResponse from @ngrx/operators inside every effect(). It is the right tool when state is too complex for plain signals but too local to justify global NgRx.

Syntax at a glance:

```
import { ComponentStore } from '@ngrx/component-store'; import { tapResponse } from
'@ngrx/operators'; // NOT from @ngrx/component-store @Injectable() export class MyStore
extends ComponentStore<MyState> { // Constructor injection required — inject() cannot be used
before super() constructor(private readonly myService: MyService) { super(initialState); }
readonly items$ = this.select(state => state.items); // Selector readonly setFilter =
this.updater((state, filter: string) => ({ ...state, filter })); // Updater readonly load =
this.effect((trigger$: Observable<void>) => // Effect trigger$.pipe( switchMap(() =>
this.myService.getItems().pipe( tapResponse( items => this.patchState({ items }), (err:
Error) => this.patchState({ error: err.message }) ) )) ) ); } @Component({ standalone: true,
providers: [MyStore], changeDetection: ChangeDetectionStrategy.OnPush }) export class
MyComponent implements OnInit { readonly #store = inject(MyStore); vm$ = this.#store.vm$;
ngOnInit(): void { this.#store.load(); } }
```

Task	API
Partial state update (no payload)	patchState({ loading: true })
Reusable mutation with typed payload	updater((state, payload) => newState)
Memoised derived Observable	select(state => ...) or select(s1$, s2$, projector)
Side effect with Observable	effect(trigger$ => trigger$.pipe(...))
Read current state snapshot	this.get() or this.get(state => state.slice)

When to use: State with 3+ interdependent slices or at least one async side effect, scoped to one component tree.

When not to use: Anything outside this component needs the state — use global NgRx store instead. State is one or two simple flags — use plain signals.

The one rule that matters most: Always use tapResponse from @ngrx/operators inside every effect() — one unhandled HTTP error permanently kills the effect for the component's lifetime with no warning and no recovery.

TOPIC 56 — NgRx SignalStore (@ngrx/signals)

Consolidated | Difficulty: Intermediate | Relevance: High

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — SignalStore introduces a new composition API (withState, withComputed, withMethods, withHooks) that feels different from both Component Store and global NgRx, but the underlying concepts — local state, computed derivations, and side effects — are ones you already know from Signals (Topic 22) and Component Store (Topic 55).

Angular Relevance: High — SignalStore is the NgRx team's Signal-native answer to Component Store, designed specifically for Angular 17+ apps built on Signals and ChangeDetectionStrategy.OnPush; it eliminates the RxJS-Observable layer entirely for state management in components that have already committed to the Signal model.

Section 1 — Explanation

The LEGO Block Analogy

Imagine building with LEGO. The old approach (Component Store) gives you a pre-shaped base plate — a fixed structure you must build on top of, following its dimensions. It works well, but you're always working within the plate's constraints.

SignalStore gives you individual LEGO bricks that snap together in any combination: withState(...) — the brick that holds your raw state values; withComputed(...) — the brick that derives new values from state; withMethods(...) — the brick that adds your updaters and effects; withHooks(...) — the brick that taps into the store's lifecycle; Custom feature bricks you or the community build and snap in anywhere.

Each brick is independently composable. You snap together only the bricks you need, in the order that makes sense for your feature. The store is the result of all the bricks snapped together — a plain Angular injectable with Signal-based properties.

Technically:

@ngrx/signals provides a signalStore() factory function. Unlike Component Store's class extension, SignalStore uses function composition — you call signalStore() with a list of 'features' and it returns an Angular injectable class. The resulting store is a class you provide and inject like any other Angular service. Its state slices are Signals — not Observables. No async pipe, no subscription management.

	Global NgRx Store	Component Store	SignalStore
API style	Class + decorators	Class extension	Function composition
State type	Plain objects	Plain objects	Angular Signals
Read state	store.select() -> Observable	select() -> Observable	store.slice() -> Signal

	Global NgRx Store	Component Store	SignalStore
Write state	Dispatch action	updater() / patchState()	patchState(store, {...})
Template binding	async pipe or selectSignal	async pipe	Direct Signal call store.count()
Lifecycle hooks	n/a	n/a	withHooks({ onInit, onDestroy })

Section 2 — Connection to Prior Concepts

Directly builds on:

- Topic 22 — Angular Signals: SignalStore's state slices are Angular Signals — store.count() is a Signal call, withComputed uses computed(), and the reactivity model is identical to Topic 22's signal/computed/effect trio.

- Topic 55 — NgRx Component Store: SignalStore solves the same problem as Component Store (local, component-scoped state) but replaces the Observable/RxJS API with Signals. Every concept maps across: updater() -> patchState(), select() -> withComputed(), effect() -> withMethods() + rxMethod().

- Topic 23 — OnPush Change Detection: SignalStore pairs perfectly with OnPush because Signals notify Angular's runtime directly — no Zone.js, no CD cycles triggered by Observable emissions.

- Topic 25 — Services & Dependency Injection: A SignalStore is a plain Angular injectable — provided in providers: [] for component scope or { providedIn: 'root' } for global scope.

- Topic 53 — NgRx Selectors In Depth: withComputed() produces computed() signals — the same memoisation rules from Topic 53 apply: new object/array literals in the computation function break memoisation.

Most directly extends: Topic 55 — NgRx Component Store.

Section 3 — Code Example

Plain TS — SignalStore concept in isolation

```
import { signal, computed } from '@angular/core'; const count = signal(0); const doubled =
computed(() => count() * 2); // Memoised derived value function increment(): void {
count.update(c => c + 1); } // SignalStore wraps this pattern into a composable, injectable
service // patchState(store, changes) replaces signal.set() for multi-slice state
```

Full Angular SignalStore Feature — User Management with HTTP

```
// user.store.ts import { signalStore, withState, withComputed, withMethods, withHooks,
patchState } from '@ngrx/signals'; import { rxMethod } from '@ngrx/signals/rxjs-interop';
import { tapResponse } from '@ngrx/operators'; import { computed, inject } from
'@angular/core'; import { pipe, switchMap, tap, debounceTime, distinctUntilChanged } from
'rxjs'; import { UserService } from './user.service'; import { User } from './user.model';
interface UserState { users: User[]; selectedUserId: string | null; loading: boolean; error:
string | null; filter: string; } const initialState: UserState = { users: [], selectedUserId:
null, loading: false, error: null, filter: '', }; export const UserStore = signalStore(
withState(initialState), // BRICK 1: Each key becomes a read-only Signal on the store
withComputed((store) => ({ // BRICK 2a: Base computed Signals — define dependencies BEFORE
dependents filteredUsers: computed(() => { const filter = store.filter().toLowerCase();
return filter ? store.users().filter(u => u.name.toLowerCase().includes(filter)) :
store.users(); }), selectedUser: computed(() => store.users().find(u => u.id ===
store.selectedUserId()) ?? null), totalUsers: computed(() => store.users().length), hasError:
computed(() => store.error() !== null), })), withComputed((store) => ({ // BRICK 2b: Second
withComputed — filteredUsers now available on store vm: computed(() => ({ users:
store.filteredUsers(), // safe — defined in brick 2a loading: store.loading(), error:
store.error(), total: store.totalUsers(), })), })), withMethods((store, userService =
inject(UserService)) => ({ // BRICK 3: Methods setFilter(filter: string): void {
patchState(store, { filter, selectedUserId: null }); }, selectUser(id: string | null): void {
patchState(store, { selectedUserId: id }); }, clearError(): void { patchState(store, { error:
null }); }, // rxMethod: bridges RxJS into the Signal world // Accepts a plain value, a
Signal, or an Observable loadUsers: rxMethod<void>(pipe( tap(() => patchState(store, {
loading: true, error: null })), switchMap(() => userService.getAll().pipe( tapResponse({
next: (users) => patchState(store, { users, loading: false }), error: (err: Error) =>
patchState(store, { error: err.message, loading: false }), }) )) )), search:
rxMethod<string>(pipe( debounceTime(300), distinctUntilChanged(), tap(() => patchState(store,
{ loading: true })), switchMap((query) => userService.search(query).pipe( tapResponse({ next:
(users) => patchState(store, { users, loading: false }), error: (err: Error) =>
patchState(store, { error: err.message, loading: false }), }) )) )), })), withHooks({ // BRICK
4: Lifecycle hooks onInit(store): void { store.loadUsers(); store.search(store.filter); //
Pass Signal, not value — reactive binding }, }) );
```

```
// users.component.ts @Component({ selector: 'app-users', standalone: true, changeDetection:
ChangeDetectionStrategy.OnPush, providers: [UserStore], // component-scoped template: ` @if
(store.loading()) { <p>Loading...</p> } @if (store.hasError()) { <p class="error">{{
store.error() }}</p> <button (click)="store.clearError()">Dismiss</button> } <input
[value]="store.filter()" (input)="store.setFilter($any($event.target).value)" /> <p>Showing
{{ store.filteredUsers().length }} of {{ store.totalUsers() }}</p> @for (user of
store.filteredUsers(); track user.id) { <div [class.selected]="user.id ===
store.selectedUserId()" (click)="store.selectUser(user.id)"> {{ user.name }} </div> } ` })
export class UsersComponent { readonly store = inject(UserStore); // public readonly —
template needs access // No ngOnInit — withHooks.onInit fires automatically // No async pipe —
Signals read directly: store.loading() }
```

Section 4 — Before & After Refactor

Before — Component Store with Observable-based template binding

```
// BEFORE: Component Store (Topic 55 style) @Injectable() export class UserStore extends
ComponentStore<UserState> { readonly vm$ = this.select( this.select(s => s.users),
this.select(s => s.loading), this.select(s => s.filter), (users, loading, filter) => ({
users, loading, filter }) ); readonly loadUsers = this.effect((trigger$: Observable<void>) =>
trigger$.pipe(switchMap(() => { this.patchState({ loading: true }); return
this.userService.getAll().pipe(tapResponse( users => this.patchState({ users, loading: false
}), (err: Error) => this.patchState({ error: err.message, loading: false }) )); })) ); }
@Component({ providers: [UserStore], template: ` @if (vm$ | async; as vm) { @if (vm.loading) {
<p>Loading...</p> } @for (user of vm.users; track user.id) { ... } }` }) export class
UsersComponent implements OnInit { vm$ = this.store.vm$; ngOnInit() { this.store.loadUsers();
} }
```

What this requires: async pipe, ngOnInit to trigger loading, Observable mental model throughout the template, and RxJS knowledge to understand the effect chain.

After — SignalStore with Signal-based template binding

```
// AFTER: SignalStore export const UserStore = signalStore( withState(initialState),
withComputed(store => ({ filteredUsers: computed(() => /* ... */) })), withMethods((store,
userService = inject(UserService)) => ({ loadUsers: rxMethod<void>(pipe( switchMap(() =>
userService.getAll().pipe(tapResponse({ next: users => patchState(store, { users, loading:
false }), error: (err: Error) => patchState(store, { error: err.message, loading: false }),
})))  )), })), withHooks({ onInit: store => store.loadUsers() }) ); @Component({ standalone:
true, providers: [UserStore], changeDetection: ChangeDetectionStrategy.OnPush, template: `
@if (store.loading()) { <p>Loading...</p> } @for (user of store.filteredUsers(); track
user.id) { ... } ` }) export class UsersComponent { readonly store = inject(UserStore); // No
lifecycle hooks, no subscriptions, no async pipe }
```

Why: No async pipe means no null-check wrapper cluttering the template. withHooks.onInit replaces ngOnInit. Signal reads are synchronous. ChangeDetectionStrategy.OnPush integrates naturally — Signals notify Angular directly without Zone.js.

Section 5 — Common Mistakes

Mistake 1 — Forward-referencing a computed Signal in the same withComputed block

```
// WRONG — vm references filteredUsers defined BELOW it withComputed((store) => ({ vm:
computed(() => ({ users: store.filteredUsers() })), // filteredUsers doesn't exist yet
filteredUsers: computed(() => store.users().filter(...)), })), // CORRECT — split into two
withComputed calls withComputed((store) => ({ filteredUsers: computed(() =>
store.users().filter(...)), })), withComputed((store) => ({ vm: computed(() => ({ users:
store.filteredUsers() })), // safe — defined above })),
```

Angular consequence: Runtime error — store.filteredUsers is not a function — because filteredUsers hasn't been added to the store object at the point vm is evaluated.

Mistake 2 — Calling patchState directly from a component

```
// WRONG — component directly mutates store state import { patchState } from '@ngrx/signals';
export class UsersComponent { readonly store = inject(UserStore); onSelect(id: string): void
{ patchState(this.store, { selectedUserId: id }); } // bypasses withMethods // CORRECT — all
mutations through withMethods // In store: withMethods((store) => ({ selectUser(id: string) {
patchState(store, { selectedUserId: id }); } })) // In component: this.store.selectUser(id)
```

Angular consequence: At team scale, mutations scattered across components make state changes unpredictable with no audit trail.

Mistake 3 — Passing the Signal value instead of the Signal to rxMethod

```
// WRONG — passes the current value once, not the Signal withHooks({ onInit(store): void {
store.search(store.filter()); } }), // () = reads value NOW // CORRECT — pass the Signal
itself — rxMethod auto-subscribes withHooks({ onInit(store): void {
store.search(store.filter); } }), // no () — passes the Signal // Every time store.filter()
changes, the search pipe re-runs automatically
```

Angular consequence: rxMethod receives a one-time string value. The search fires once on init and never again when the filter changes — reactive binding completely lost.

Section 6 — Do's and Don'ts

DO	DON'T
Define computed Signals in dependency order — dependencies before dependents	Forward-reference a computed defined later in the same withComputed() object literal
Put all patchState() calls inside withMethods()	Call patchState(store, ...) directly from a component — bypasses encapsulation
Pass a Signal (not its value) to rxMethod: store.search(store.filter)	Pass store.filter() (with ()) to rxMethod — fires once, not reactively
Use withHooks({ onInit }) for initial data loading	Trigger initial loads in ngOnInit when the store can self-initialise
Use rxMethod() for any method involving Observables, HTTP, or debouncing	Use plain async/await for complex Observable chains — you lose switchMap cancellation
Use tapResponse from @ngrx/operators inside every rxMethod pipe	Use plain tap() with no error handling — permanently kills the method on first error

Section 7 — Interview Prep

Question 1

"What is NgRx SignalStore and how does it differ from NgRx Component Store? Which would you choose for a new Angular 17+ project?"

Strong answer covers:

- Component Store: class extension, Observable-based (select() -> Observable, effect() -> Observable chain), requires async pipe in templates
- SignalStore: function composition (signalStore() + withX() bricks), Signal-based (state slices are Signals, withComputed uses computed()), no async pipe needed
- SignalStore integrates more naturally with Angular 17+'s Signal-based reactivity and ChangeDetectionStrategy.OnPush
- For new projects on Angular 17+: SignalStore; for existing projects already using Component Store: no urgent need to migrate

Weak answers miss: The Observable vs Signal distinction — most candidates describe the API difference but not why Signals eliminate the async pipe and subscription management complexity.

Level: Mid

Question 2

"Explain rxMethod in NgRx SignalStore. What inputs does it accept, and what problem does it solve that a plain async method cannot?"

Strong answer covers:

- rxMethod<T>() wraps an RxJS pipe and returns a method that accepts a plain value, a Signal, or an Observable

- When passed a Signal, it auto-subscribes and re-runs the pipe on every Signal emission — reactive binding without manual subscription

- The pipe can use debounceTime, switchMap, concatMap — full RxJS operator semantics including request cancellation

- A plain async method cannot cancel in-flight requests, debounce, or reactively bind to a Signal

- Subscriptions created by rxMethod are automatically cleaned up when the store is destroyed

Weak answers miss: The Signal-as-input feature — most candidates explain rxMethod as an HTTP helper but miss that passing store.query (a Signal) creates a fully reactive pipeline.

Level: Senior

Section 8 — Quick Reference Card

3-line definition: NgRx SignalStore composes feature bricks (withState, withComputed, withMethods, withHooks) into an Angular injectable whose state slices are read-only Signals — consumed directly in templates without async pipe, subscriptions, or Observable ceremony. All mutations go through withMethods() via patchState(store, partial). RxJS interop is handled by rxMethod() from @ngrx/signals/rxjs-interop, which accepts plain values, Signals, or Observables and manages subscriptions automatically.

Syntax at a glance:

```
export const MyStore = signalStore( withState({ items: [], loading: false, filter: '' }), withComputed((store) => ({ filteredItems: computed(() => store.items().filter(i => i.name.includes(store.filter()))), })), withMethods((store, service = inject(MyService)) => ({ setFilter(filter: string): void { patchState(store, { filter }); }, loadItems: rxMethod<void>(pipe( // from @ngrx/signals/rxjs-interop tap(() => patchState(store, { loading: true })), switchMap(() => service.getAll().pipe( tapResponse({ // from @ngrx/operators next: items => patchState(store, { items, loading: false }), error: (e: Error) => patchState(store, { loading: false }), }) )) )), })), withHooks({ onInit(store): void { store.loadItems(); } }) ); @Component({ standalone: true, providers: [MyStore], changeDetection: ChangeDetectionStrategy.OnPush, template: ` @if (store.loading()) { <p>Loading...</p> } @for (item of store.filteredItems(); track item.id) { <div>{{ item.name }}</div> } ` }) export class MyComponent { readonly store = inject(MyStore); // public readonly for template access }
```

Task	API
Define raw state	withState({ key: initialValue })
Derive computed values	withComputed(store => ({ name: computed(() => ...) }))
Add mutations and effects	withMethods((store) => ({ method() { patchState(store, ...) } }))
HTTP / Observable side effect	rxMethod<T>(pipe(switchMap(...), tapResponse(...)))
Lifecycle (load on init)	withHooks({ onInit(store) { store.load(); } })

Task	API
Read state in template	store.slice() — Signal call, no async pipe

When to use: Angular 17+ projects committed to the Signal model; component-scoped state with 3+ interdependent slices or async effects; when eliminating async pipe and subscription management is a priority.

When not to use: State needed across unrelated features — use global NgRx store. Simple one or two-flag state — use raw signal() and computed(). Teams not yet on Angular 17+.

The one rule that matters most: All patchState() calls belong inside withMethods() — exposing state mutation through named methods is what makes a SignalStore auditable, testable, and maintainable at team scale.

TOPIC 57 — NgRx Router Store

Consolidated | Difficulty: Intermediate | Relevance: High

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — Router Store introduces one new concept (connecting the Angular Router to NgRx) but relies entirely on patterns you already know: selectors, effects, and the global store. The new API surface is small; the value is architectural.

Angular Relevance: High — In any NgRx application where components read route params, query params, or the current URL to drive data loading, Router Store eliminates the direct ActivatedRoute injection inside components and makes the URL a first-class, selectable, testable slice of your NgRx state.

Section 1 — Explanation

The GPS Tracker Analogy

Imagine a logistics company. Every truck has a GPS unit broadcasting its position. Without a central system, each warehouse manager calls the driver directly to ask 'where are you?' — direct, coupled, hard to monitor or replay.

With a central dispatch system, every GPS update is logged to a shared board. Any manager reads the board. The operations team can replay the route history. No one calls the driver directly.

NgRx Router Store is the dispatch system for your URL. Without it: components inject ActivatedRoute directly and ask the router 'where am I?' — direct, coupled, hard to test, impossible to replay in DevTools. With it: every navigation event is dispatched as an NgRx action and the current router state (URL, params, query params, fragment) is stored as a slice of your NgRx state.

Router Actions:

Action	When it fires
routerRequestAction	Navigation starts (before guards run)
routerNavigationAction	Guards passed — navigation is committing
routerNavigatedAction	Navigation fully complete — component activated
routerCancelAction	A guard cancelled the navigation
routerErrorAction	Navigation threw an error

Section 2 — Connection to Prior Concepts

Directly builds on:

- Topic 32 — Angular Routing Fundamentals: Router Store wraps the Angular Router's event system — you need to understand ActivatedRoute, route params, query params, and navigation events to know what Router Store is exposing to NgRx.

- Topic 51 — NgRx Core Concepts: Router state is stored as a reducer slice in the NgRx store — the same createFeatureSelector and createSelector pattern from Topic 51 applies to reading it.

- Topic 52 — NgRx Effects: The most powerful Router Store pattern is Effects that listen to routerNavigatedAction and dispatch HTTP load actions — replacing the ActivatedRoute + switchMap pattern with a fully NgRx-managed data loading pipeline.

- Topic 53 — NgRx Selectors In Depth: Router Store provides built-in selectors, but composing your own feature selectors on top of them follows the exact layered selector pattern from Topic 53.

- Topic 35 — Route Guards: Router Store's routerCancelAction fires when a guard blocks navigation — Effects can react to cancellations for analytics or UX feedback.

Bridging note: In Topic 32 you used route.paramMap.pipe(switchMap(...), takeUntilDestroyed()) to react to route param changes inside a component. Router Store moves that reaction into an Effect: the Effect listens to routerNavigatedAction, extracts the param from the store via a selector, and dispatches a load action — the component never touches ActivatedRoute at all.

Most directly extends: Topic 32 — Angular Routing Fundamentals + Topic 52 — NgRx Effects.

Section 3 — Code Example

Setup — registering Router Store

```
// app.config.ts import { ApplicationConfig } from '@angular/core'; import { provideRouter } from '@angular/router'; import { provideStore } from '@ngrx/store'; import { provideRouterStore, routerReducer } from '@ngrx/router-store'; import { provideStoreDevtools } from '@ngrx/store-devtools'; import { CustomRouterStateSerializer } from './custom-route-serializer'; export const appConfig: ApplicationConfig = { providers: [ provideRouter(routes), provideStore({ router: routerReducer }), // Register routerReducer under 'router' key provideRouterStore({ serializer: CustomRouterStateSerializer }), // ALWAYS use custom serializer provideStoreDevtools({ maxAge: 25, logOnly: false }), ], };
```

Custom Router State Serializer — always required in production

```
// custom-route-serializer.ts import { RouterStateSnapshot, Params } from '@angular/router'; import { RouterStateSerializer } from '@ngrx/router-store'; export interface CustomRouterState { url: string; params: Params; queryParams: Params; data: Record<string, unknown>; } export class CustomRouterStateSerializer implements RouterStateSerializer<CustomRouterState> { serialize(routerState: RouterStateSnapshot): CustomRouterState { let route = routerState.root; while (route.firstChild) { route = route.firstChild; } // Walk to deepest route return { url: routerState.url, params: route.params, queryParams: routerState.root.queryParams, data: route.data as Record<string, unknown>, }; } }
```

Built-in Router Selectors + Custom composition

```
// router.selectors.ts import { createSelector } from '@ngrx/store'; import {
getRouterSelectors } from '@ngrx/router-store'; import { selectUserEntities } from
'./user.selectors'; export const { selectUrl, selectQueryParams, selectRouteParams,
selectRouteData } = getRouterSelectors(); // Derive typed param from the raw string
dictionary export const selectUserId = createSelector( selectRouteParams, (params) =>
params['id'] as string | undefined ); // Cross-slice selector — combines router state + entity
state (Topics 53 + 54) export const selectCurrentUser = createSelector( selectUserEntities,
selectUserId, (entities, id) => (id ? entities[id] ?? null : null) );
```

Effect that reacts to navigation — the core Router Store pattern

```
// user.effects.ts import { Injectable, inject } from '@angular/core'; import { Actions,
createEffect, ofType, concatLatestFrom } from '@ngrx/effects'; import { Store } from
'@ngrx/store'; import { routerNavigatedAction } from '@ngrx/router-store'; import {
switchMap, map, filter } from 'rxjs'; import { catchError, of } from 'rxjs'; import {
selectUserId } from './router.selectors'; @Injectable() export class UserEffects { readonly
#actions$ = inject(Actions); readonly #store = inject(Store); readonly #userService =
inject(UserService); // concatLatestFrom reads state AFTER the action has been reduced // Use
concatLatestFrom (not withLatestFrom) for router action effects loadUserOnNavigation$ =
createEffect(() => this.#actions$.pipe( ofType(routerNavigatedAction), concatLatestFrom(() =>
this.#store.select(selectUserId)), filter(([, userId]) => userId !== undefined),
switchMap(([, userId]) => this.#userService.getById(userId!).pipe( map(user =>
UsersApiActions.userLoaded({ user })), catchError(error =>
of(UsersApiActions.usersLoadFailed({ error: error.message })))  ) ) ) ); }
```

Component — reads route-derived state from the store, never from ActivatedRoute

```
// user-detail.component.ts @Component({ selector: 'app-user-detail', standalone: true,
changeDetection: ChangeDetectionStrategy.OnPush, template: ` @if (loading()) {
<p>Loading...</p> } @else if (user()) { <h1>{{ user()!.name }}</h1><p>{{ user()!.email }}</p>
} @else { <p>User not found</p> } ` }) export class UserDetailComponent { readonly #store =
inject(Store); // Cross-slice selector combines router params + entity state user =
this.#store.selectSignal(selectCurrentUser); // No ActivatedRoute injection loading =
this.#store.selectSignal(selectLoading); // No ngOnInit — Effect handles loading when
routerNavigatedAction fires }
```

Section 4 — Before & After Refactor

Before — Direct ActivatedRoute injection in the component

```
// WRONG: component coupled to router directly @Component({}) export class
UserDetailComponent implements OnInit { readonly #route = inject(ActivatedRoute); readonly
#userService = inject(UserService); readonly #destroyRef = inject(DestroyRef); user: User |
null = null; loading = false; ngOnInit(): void { this.#route.paramMap.pipe( switchMap(params
=> { const id = params.get('id')!; this.loading = true; return this.#userService.getById(id);
}), takeUntilDestroyed(this.#destroyRef) ).subscribe({ next: user => { this.user = user;
this.loading = false; }, error: () => { this.loading = false; } }); } }
```

What breaks: Component owns HTTP logic — cannot be tested without a real router and real HTTP. State is local. Navigation invisible to DevTools. If two components need the same current user, both subscribe to ActivatedRoute separately — two HTTP calls.

After — Router Store + Effect pattern

```
// CORRECT: Component is a pure view @Component({ standalone: true, changeDetection:
ChangeDetectionStrategy.OnPush }) export class UserDetailComponent { readonly #store =
inject(Store); user = this.#store.selectSignal(selectCurrentUser); loading =
this.#store.selectSignal(selectLoading); // No ActivatedRoute, no HTTP, no subscriptions, no
ngOnInit }
```

Why: The component is now a pure view — testable with just a MockStore. The loaded user is in the global store — any other component can select it without a second HTTP call. DevTools shows the full chain: [Router] Navigated -> [Users API] User Loaded. The Effect handles switchMap cancellation automatically.

Section 5 — Common Mistakes

Mistake 1 — Forgetting to register routerReducer

```
// WRONG — provideRouterStore() without routerReducer provideStore({ users: usersReducer }),
// No 'router' key provideRouterStore(), // Connected but has nowhere to write state //
CORRECT provideStore({ router: routerReducer, users: usersReducer }), provideRouterStore({
serializer: CustomRouterStateSerializer }),
```

Angular consequence: All router selectors return undefined silently. Effects using concatLatestFrom(store.select(selectRouteParams)) return undefined — Effects produce malformed actions with no error.

Mistake 2 — Using routerNavigationAction when you need routerNavigatedAction

```
// WRONG — fires before the component is activated ofType(routerNavigationAction), // Guards
haven't completed — component not mounted // CORRECT — navigation fully complete, component is
active ofType(routerNavigatedAction), concatLatestFrom(() =>
this.#store.select(selectUserId)),
```

Angular consequence: If a guard delays or cancels navigation, routerNavigationAction may fire before the final route is confirmed — loading data for a route the user never reaches.

Mistake 3 — Using withLatestFrom instead of concatLatestFrom for router state in Effects

```
// WRONG — timing race: store may not have processed the router action yet
ofType(routerNavigatedAction), withLatestFrom(this.#store.select(selectUserId)), // May read
PREVIOUS route's userId // CORRECT — concatLatestFrom reads state AFTER the action is reduced
import { concatLatestFrom } from '@ngrx/effects'; ofType(routerNavigatedAction),
concatLatestFrom(() => this.#store.select(selectUserId)), // Guaranteed new route's userId
```

Angular consequence: Navigating from /users/42 to /users/99 — the Effect fires but selectUserId returns '42' because the router reducer hasn't processed routerNavigatedAction yet. Wrong user's data is loaded.

Section 6 — Do's and Don'ts

DO	DON'T
Register routerReducer in provideStore({ router: routerReducer }) alongside provideRouterStore()	Call provideRouterStore() without registering routerReducer — selectors return undefined
Always write a custom RouterStateSerializer extracting only url, params, queryParams, data	Use the default serializer in production — stores un-serialisable Angular objects, breaks DevTools
Use routerNavigatedAction for data-loading Effects	Use routerNavigationAction for data loading — component isn't mounted yet, guards may cancel

DO	DON'T
Use concatLatestFrom (not withLatestFrom) when reading router state inside router action Effects	Use withLatestFrom for router state in router action Effects — causes timing race with the router reducer
Compose typed selectors on top of getRouterSelectors() — extract and type-cast params in one place	Read selectRouteParams directly in components and cast inline — cast logic scattered everywhere
React to route changes in Effects (ofType(routerNavigatedAction))	Inject ActivatedRoute in components alongside NgRx — creates dual source of truth

Section 7 — Interview Prep

Question 1

"What is NgRx Router Store, what problem does it solve, and how do you set it up?"

Strong answer covers:

- Problem: ActivatedRoute in components creates a dual source of truth alongside NgRx state; Effects have no clean way to react to navigation; route state is invisible to DevTools
- Solution: Router Store dispatches navigation as NgRx actions and stores route state (url, params, queryParams) as a selectable NgRx slice
- Setup: provideStore({ router: routerReducer }) + provideRouterStore({ serializer: CustomRouterStateSerializer })
- Selectors: getRouterSelectors() provides selectUrl, selectRouteParams, selectQueryParams, selectRouteData
- Primary pattern: Effects listen to routerNavigatedAction + concatLatestFrom(selectRouteParam) to load data on navigation

Weak answers miss: The custom serializer requirement — most candidates describe setup without mentioning that the default serializer breaks DevTools serialisation in production.

Level: Mid

Question 2

"Why use concatLatestFrom instead of withLatestFrom when reading router state inside an Effect that reacts to a router action?"

Strong answer covers:

- withLatestFrom reads the store at the moment the outer Observable emits — which is when routerNavigatedAction dispatches
- The router reducer processes the same action to update the store — there is a timing window where the action has dispatched but state hasn't been reduced yet
- concatLatestFrom defers the state read until after the action has been fully reduced — guaranteed to reflect the new route

- Practical symptom without concatLatestFrom: navigating from /users/42 to /users/99, the Effect reads id: '42' from the store

- concatLatestFrom is provided by @ngrx/effects specifically to solve this timing issue

Weak answers miss: The root cause — most candidates know to use concatLatestFrom as a rule but cannot explain the action-reduction timing race that makes it necessary.

Level: Senior

Section 8 — Quick Reference Card

3-line definition: NgRx Router Store logs every Angular navigation as an NgRx action and stores the current route state (url, params, queryParams) as a selectable NgRx slice — making the URL a first-class member of your application state. Effects replace direct ActivatedRoute injection by reacting to routerNavigatedAction. A custom RouterStateSerializer is always required to prevent DevTools serialisation errors.

Syntax at a glance:

```
// app.config.ts provideStore({ router: routerReducer }), provideRouterStore({ serializer: CustomRouterStateSerializer }), // custom-route-serializer.ts export class CustomRouterStateSerializer implements RouterStateSerializer<CustomRouterState> { serialize(state: RouterStateSnapshot): CustomRouterState { let route = state.root; while (route.firstChild) route = route.firstChild; return { url: state.url, params: route.params, queryParams: state.root.queryParams, data: route.data as Record<string, unknown> }; } } // router.selectors.ts const { selectRouteParams, selectQueryParams } = getRouterSelectors(); export const selectUserId = createSelector(selectRouteParams, params => params['id'] as string | undefined); export const selectCurrentUser = createSelector( selectUserEntities, selectUserId, (entities, id) => id ? entities[id] ?? null : null ); // Effect loadUserOnNavigation$ = createEffect(() => this.#actions$.pipe( ofType(routerNavigatedAction), concatLatestFrom(() => this.#store.select(selectUserId)), filter(([, id]) => id !== undefined), switchMap(([, id]) => this.#userService.getById(id!).pipe( map(user => UsersApiActions.userLoaded({ user })), catchError(err => of(UsersApiActions.usersLoadFailed({ error: err.message })))) ) ); // Component — reads from store, no ActivatedRoute user = this.#store.selectSignal(selectCurrentUser); loading = this.#store.selectSignal(selectLoading);
```

Task	API
Register Router Store	provideStore({ router: routerReducer }) + provideRouterStore({ serializer })
Read current URL	getRouterSelectors().selectUrl
Read route params	getRouterSelectors().selectRouteParams
Read query params	getRouterSelectors().selectQueryParams
Load data on navigation	Effect: ofType(routerNavigatedAction) + concatLatestFrom + filter + switchMap
Prevent DevTools errors	RouterStateSerializer extracting only url, params, queryParams, data

When to use: Any NgRx app where data loading is triggered by route changes; when components need access to 'the currently viewed entity' without re-fetching; when URL-driven filter/pagination state must be selectable from the store.

The one rule that matters most: Always write a custom RouterStateSerializer — the default stores Angular component references and injector instances that cannot be JSON-serialised, breaking Redux DevTools time-travel and any strictStateSerializability runtime check.

TOPIC 58 — NgRx Testing (Store, Effects, Selectors, Component Store, SignalStore)

Consolidated | Difficulty: Advanced | Relevance: Critical

Section 0 — Difficulty & Priority Rating

Difficulty: Advanced — NgRx Testing requires you to hold five distinct testing contexts in your head simultaneously (Store, Effects, Selectors, Component Store, SignalStore), each with its own setup pattern, mock strategy, and assertion style — and the bugs it catches are the silent, production-breaking kind that unit tests alone miss.

Angular Relevance: Critical — Every NgRx pattern you have learned in Topics 51-57 is only production-safe if it is tested; untested reducers silently mutate state, untested effects silently swallow errors or run forever, and untested selectors silently recompute on every action — NgRx Testing is the verification layer that makes the entire NgRx architecture trustworthy.

Section 1 — Explanation

The Flight Simulator Analogy

A commercial airline does not let pilots fly a new aircraft type by reading the manual and then boarding 300 passengers. They spend hours in a flight simulator — a controlled environment that mimics every system of the real aircraft, lets the pilot make mistakes, and verifies the correct response to every scenario before a real flight.

NgRx Testing is your flight simulator for application state.

- MockStore is the simulator cockpit — it looks and feels like the real NgRx store, but you control every instrument reading (state slice, selector result) directly. No real reducers run, no real effects fire.
- provideMockActions is the simulated air traffic control — you feed exactly the actions you want the effect to receive, in the exact sequence, and verify what it transmits back.
- Selector projector testing is instrument calibration — you test the pure function inside the selector directly, with no store involved at all.
- ComponentStore / SignalStore testing is a ground-level systems check — you provide the real store (not a mock) with a mock service, and verify that methods, updaters, and effects produce the correct state transitions.

Layer	Primary Tool	Key Setup
Reducer	Plain function call — no Angular needed	reducer(initialState, action)
Selector	selector.projector(mockInputs)	No store, no TestBed
Store (component test)	MockStore + provideMockStore()	TestBed.configureTestingModule
Effects	provideMockActions() + ReplaySubject	TestBed + real or mock services

Layer	Primary Tool	Key Setup
Component Store	Real ComponentStore + mock service	TestBed with providers
SignalStore	Real SignalStore + mock service	TestBed with providers

Section 2 — Connection to Prior Concepts

Directly builds on:

- Topic 49 — Angular Testing Basics: fixture.detectChanges(), MockStore in TestBed, fakeAsync/tick, and data-testid querying — all patterns that now apply to NgRx-connected components.

- Topic 51 — NgRx Core Concepts: Reducers are pure functions — testing them requires no Angular setup, just reducer(state, action). Understanding the reducer contract makes the tests obvious.

- Topic 52 — NgRx Effects: Effects are the most complex NgRx layer to test — provideMockActions, ReplaySubject, and the inner/outer Observable structure are the testing surface for Topic 52's patterns.

- Topic 53 — NgRx Selectors In Depth: The selector.projector() method tests the pure derivation function directly — understanding that selectors are composed pure functions makes this approach obvious.

- Topics 55 & 56 — Component Store and SignalStore: Both are tested by providing the real store class in TestBed with mocked services — the testing pattern mirrors the DI pattern from those topics.

Bridging note: In earlier testing topics, you tested services by providing HttpClientTestingModule and flushing requests manually. In NgRx Effect tests, you do exactly that — plus you feed actions into the effect via a ReplaySubject controlled by provideMockActions. The HTTP testing layer is identical; only the trigger mechanism changes.

Most directly extends: Topic 52 — NgRx Effects (the effect structure under test) and Topic 53 — NgRx Selectors In Depth (projector testing).

Section 3 — Code Example

Testing 1 — Reducer (no Angular setup needed)

```
// user.reducer.spec.ts describe('usersReducer', () => { const state = { users: [], loading: false, error: null }; it('should return the same state reference for an unknown action', () => { const result = usersReducer(state, { type: 'UNKNOWN' } as any); expect(result).toBe(state); // toBe = reference equality — must be the SAME object }); it('should store users and clear loading on usersLoaded', () => { const users: User[] = [{ id: '1', name: 'Alice', email: 'alice@example.com' }]; const loadingState = { ...state, loading: true }; const result = usersReducer(loadingState, UsersApiActions.usersLoaded({ users })); expect(result.users).toEqual(users); // toEqual = deep equality expect(result.loading).toBe(false); expect(result).not.toBe(loadingState); // Must return a NEW object reference }); it('should store the error on usersLoadFailed', () => { const result = usersReducer(state, UsersApiActions.usersLoadFailed({ error: 'Network error' })); expect(result.error).toBe('Network error'); expect(result.loading).toBe(false); }); });
```

Testing 2 — Selectors via selector.projector()

```
// user.selectors.spec.ts describe('User Selectors', () => { const users: User[] = [ { id:
'1', name: 'Alice', email: 'alice@admin.com' }, { id: '2', name: 'Bob', email:
'bob@example.com' }, ]; it('selectAllUsers projector should return users from the state
slice', () => { // projector() calls the selector's pure function directly — no store, no
TestBed const result = selectAllUsers.projector({ users, loading: false, error: null });
expect(result).toEqual(users); }); it('selectAdminUsers projector should filter to admin
emails', () => { // selectAdminUsers depends on selectAllUsers — its projector input is
User[], not UsersState const result = selectAdminUsers.projector(users);
expect(result).toHaveSize(1); expect(result[0].name).toBe('Alice'); }); });
```

Testing 3 — NgRx Store in a component test with MockStore

```
// users.component.spec.ts import { provideMockStore, MockStore } from '@ngrx/store/testing';
describe('UsersComponent', () => { let fixture: ComponentFixture<UsersComponent>; let store:
MockStore; beforeEach(async () => { await TestBed.configureTestingModule({ imports:
[UsersComponent], providers: [provideMockStore({ initialState: { users: { users: [], loading:
false, error: null } } })], }).compileComponents(); store = TestBed.inject(MockStore);
fixture = TestBed.createComponent(UsersComponent); }); afterEach(() => {
store.resetSelectors(); }); // REQUIRED — clears overrides between tests it('should dispatch
pageOpened on init', () => { const dispatchSpy = spyOn(store, 'dispatch');
fixture.detectChanges();
expect(dispatchSpy).toHaveBeenCalledWith(UsersPageActions.pageOpened()); }); it('should
render users from the store', () => { store.overrideSelector(selectAllUsers, mockUsers);
store.overrideSelector(selectLoading, false); store.refreshState(); // Apply overrides — emit
new state fixture.detectChanges(); // Update DOM const items =
fixture.nativeElement.querySelectorAll('[data-testid="user-name"]');
expect(items).toHaveSize(1); expect(items[0].textContent).toContain('Alice'); }); });
```

Testing 4 — NgRx Effects (mocked service)

```
// user.effects.spec.ts import { provideMockActions } from '@ngrx/effects/testing'; import {
provideMockStore } from '@ngrx/store/testing'; import { ReplaySubject, of, throwError } from
'rxjs'; import { firstValueFrom } from 'rxjs'; describe('UserEffects', () => { let effects:
UserEffects; let actions$: ReplaySubject<Action>; let userService:
jasmine.SpyObj<UserService>; beforeEach(() => { // ReplaySubject(1): replays last emission to
late subscribers // Effects subscribe to actions$ AFTER this subject emits — Subject would
miss it actions$ = new ReplaySubject<Action>(1); const spy =
jasmine.createSpyObj('UserService', ['getAll']); TestBed.configureTestingModule({ providers:
[ UserEffects, provideMockActions(() => actions$), provideMockStore(), // Required if Effect
reads store via concatLatestFrom { provide: UserService, useValue: spy }, ], }); effects =
TestBed.inject(UserEffects); userService = TestBed.inject(UserService) as
jasmine.SpyObj<UserService>; }); it('should dispatch usersLoaded on success', async () => {
userService.getAll.and.returnValue(of(mockUsers));
actions$.next(UsersPageActions.pageOpened()); // firstValueFrom converts Observable to
Promise — safe async assertion const action = await firstValueFrom(effects.loadUsers$);
expect(action).toEqual(UsersApiActions.usersLoaded({ users: mockUsers })); }); it('should
dispatch usersLoadFailed on HTTP error', async () => {
userService.getAll.and.returnValue(throwError(() => new Error('Network error')));
actions$.next(UsersPageActions.pageOpened()); const action = await
firstValueFrom(effects.loadUsers$); expect(action).toEqual(UsersApiActions.usersLoadFailed({
error: 'Network error' })); }); it('should remain alive after an error and handle a subsequent
action', async () => { userService.getAll.and.returnValue(throwError(() => new
Error('fail'))); actions$.next(UsersPageActions.pageOpened()); await
firstValueFrom(effects.loadUsers$); // Error path — effect must survive
userService.getAll.and.returnValue(of(mockUsers));
actions$.next(UsersPageActions.pageOpened()); const action = await
firstValueFrom(effects.loadUsers$); expect(action).toEqual(UsersApiActions.usersLoaded({
users: mockUsers })); }); });
```

Testing 5 — Effects with real HTTP interception (Angular 18+)

```
import { provideHttpClient } from '@angular/common/http'; import { provideHttpClientTesting, HttpTestingController } from '@angular/common/http/testing'; describe('UserEffects with real HTTP', () => { let effects: UserEffects; let actions$: ReplaySubject<Action>; let httpController: HttpTestingController; beforeEach(() => { actions$ = new ReplaySubject<Action>(1); TestBed.configureTestingModule({ providers: [ provideHttpClient(), // Angular 18+ functional HTTP provider provideHttpClientTesting(), // Replaces real HTTP with fake backend UserEffects, UserService, provideMockActions(() => actions$), provideMockStore(), ], }); effects = TestBed.inject(UserEffects); httpController = TestBed.inject(HttpTestingController); }); afterEach(() => { httpController.verify(); }); // Topic 48 rule — no unexpected requests it('should call GET /api/users and dispatch usersLoaded', (done) => { actions$.next(UsersPageActions.pageOpened()); effects.loadUsers$.subscribe(action => { expect(action).toEqual(UsersApiActions.usersLoaded({ users: [{ id: '1', name: 'Alice', email: '' }] })); done(); }); const req = httpController.expectOne('/api/users'); expect(req.request.method).toBe('GET'); req.flush([{ id: '1', name: 'Alice', email: '' }]); }); });
```

Testing 6 — NgRx Component Store

```
// data-table.store.spec.ts describe('DataTableStore', () => { let store: DataTableStore; let userService: jasmine.SpyObj<UserService>; beforeEach(() => { const spy = jasmine.createSpyObj('UserService', ['getAll', 'search']); TestBed.configureTestingModule({ providers: [DataTableStore, { provide: UserService, useValue: spy }], }); store = TestBed.inject(DataTableStore); userService = TestBed.inject(UserService) as jasmine.SpyObj<UserService>; }); it('should initialise with empty state', () => { expect(store.users()).toEqual([]); // Signal read — synchronous expect(store.loading()).toBe(false); expect(store.error()).toBeNull(); }); it('setFilter updater should update filter', () => { store.setFilter('alice'); expect(store.filter()).toBe('alice'); // Updaters are synchronous }); it('should populate users after loadUsers effect succeeds', fakeAsync(() => { userService.getAll.and.returnValue(of(mockUsers)); store.loadUsers(); tick(); // Flush pending microtasks expect(store.users()).toEqual(mockUsers); expect(store.loading()).toBe(false); })); });
```

Testing 7 — NgRx SignalStore

```
// user.store.spec.ts describe('UserStore (SignalStore)', () => { let store: InstanceType<typeof UserStore>; let userService: jasmine.SpyObj<UserService>; beforeEach(() => { const spy = jasmine.createSpyObj('UserService', ['getAll']); // Configure spy BEFORE TestBed.inject — withHooks.onInit fires during injection spy.getAll.and.returnValue(of([])); // Return empty to prevent state pollution TestBed.configureTestingModule({ providers: [UserStore, { provide: UserService, useValue: spy }], }); store = TestBed.inject(UserStore); // onInit fires here userService = TestBed.inject(UserService) as jasmine.SpyObj<UserService>; }); it('should initialise with empty state', () => { expect(store.users()).toEqual([]); expect(store.loading()).toBe(false); }); it('loadUsers should populate users on success', fakeAsync(() => { userService.getAll.and.returnValue(of(mockUsers)); store.loadUsers(); tick(); expect(store.users()).toEqual(mockUsers); })); it('setFilter should update filter state', () => { store.setFilter('alice'); expect(store.filter()).toBe('alice'); }); });
```

Section 4 — Before & After Refactor

Before — Testing a component with real Store and real reducers

```
// WRONG: uses real Store with real reducers beforeEach(async () => { await TestBed.configureTestingModule({ imports: [UsersComponent, StoreModule.forRoot({ users: usersReducer })], }).compileComponents(); store = TestBed.inject(Store); }); it('should show users', () => { store.dispatch(UsersApiActions.usersLoaded({ users: mockUsers })); // Must dispatch real action fixture.detectChanges(); // Tests reducer AND component simultaneously — no isolation });
```

What breaks: The test is testing reducer + component simultaneously. Setting up complex initial state requires dispatching multiple actions in sequence. Testing edge-case rendering (loading: true with users: [...] simultaneously) is impossible without breaking the reducer contract.

After — MockStore with overrideSelector

```
// CORRECT: MockStore bypasses reducers entirely beforeEach(async () => { await
TestBed.configureTestingModule({ imports: [UsersComponent], providers: [provideMockStore({
initialState: {} })], }).compileComponents(); store = TestBed.inject(MockStore); });
afterEach(() => { store.resetSelectors(); }); it('should show users', () => {
store.overrideSelector(selectAllUsers, mockUsers); store.overrideSelector(selectLoading,
false); store.refreshState(); fixture.detectChanges(); const items =
fixture.nativeElement.querySelectorAll('[data-testid="user-item"]');
expect(items).toHaveSize(mockUsers.length); });
```

Why: Component tests now test only rendering and event dispatching. Any state combination can be injected directly. Tests run faster. store.resetSelectors() in afterEach ensures overrides don't leak.

Section 5 — Common Mistakes

Mistake 1 — Asserting inside subscribe() without done — the silent green test

```
// WRONG — assertion never runs, test is permanently green it('should dispatch usersLoaded',
() => { // No done parameter userService.getAll.and.returnValue(of(mockUsers));
actions$.next(UsersPageActions.pageOpened()); effects.loadUsers$.subscribe(action => { //
Jasmine exits before this runs — assertion NEVER EXECUTES
expect(action.type).toBe(UsersApiActions.usersLoaded.type); }); }); // CORRECT —
firstValueFrom converts to Promise for safe async assertion it('should dispatch usersLoaded',
async () => { userService.getAll.and.returnValue(of(mockUsers));
actions$.next(UsersPageActions.pageOpened()); const action = await
firstValueFrom(effects.loadUsers$); expect(action).toEqual(UsersApiActions.usersLoaded({
users: mockUsers })); });
```

Angular consequence: Your entire Effects test suite appears green but tests nothing — every assertion inside every subscribe() callback is silently skipped. This is the single most dangerous NgRx testing bug.

Mistake 2 — Forgetting store.resetSelectors() in afterEach

```
// WRONG — override leaks into next test it('test A', () => {
store.overrideSelector(selectAllUsers, [mockUser]); store.refreshState(); }); it('test B', ()
=> { store.refreshState(); fixture.detectChanges(); // selectAllUsers STILL returns
[mockUser] from test A — silently wrong }); // CORRECT afterEach(() => {
store.resetSelectors(); }); // Clears ALL overrideSelector calls
```

Angular consequence: Tests become order-dependent — they pass in isolation but fail in full suite runs. Classic 'flaky tests' that hide locally and surface in CI.

Mistake 3 — Testing selector composition through MockStore instead of selector.projector()

```
// WRONG — conflates store registration with derivation logic it('should return admin users',
() => { store.setState({ users: { users: mockUsers, loading: false, error: null } }); let
result: User[] = []; store.select(selectAdminUsers).subscribe(u => result = u);
store.refreshState(); expect(result).toHaveSize(1); }); // CORRECT — test the pure function
directly it('should return admin users', () => { const result =
selectAdminUsers.projector(mockUsers); // Input is User[], not UsersState
expect(result).toHaveSize(1); });
```

Angular consequence: Slower tests, harder-to-debug failures, and TestBed overhead for what is a pure function test.

Section 6 — Do's and Don'ts

DO	DON'T
Test reducers as plain functions: reducer(state, action) — no Angular, no TestBed	Import StoreModule.forRoot() in reducer tests — reducers are pure functions
Test selector projectors via selector.projector(mockInputs) — no store needed	Test selectors by setting up MockStore state — slower and conflates concerns
Use provideMockStore() in component tests and overrideSelector() for state control	Use real StoreModule.forRoot() in component tests — forces reducer sequencing
Always call store.resetSelectors() in afterEach	Omit resetSelectors() — overrides leak between tests, causing order-dependent failures
Use firstValueFrom(), done, or fakeAsync+tick() for all Effect assertions	Assert inside subscribe() without done — Jasmine exits before callback runs
Use ReplaySubject<Action>(1) for actions$	Use Subject<Action> — Effects subscribing after emission miss the action entirely
Configure mock service before TestBed.inject(Store) for SignalStores with withHooks.onInit	Configure mock service after injection — onInit fires during injection with unprepared spy

Section 7 — Interview Prep

Question 1

"How do you test an NgRx Effect? Walk me through the setup and explain why each piece is needed."

Strong answer covers:

- ReplaySubject<Action>(1) for actions$ — replays last emission to late-subscribing Effects; Subject loses emissions if Effect subscribes after dispatch
- provideMockActions(() => actions$) — connects the ReplaySubject to the Effect's Actions injection token
- provideMockStore() — required if the Effect uses concatLatestFrom to read store state
- Mock service via { provide: RealService, useValue: spyObj } — isolates the Effect from real HTTP

- firstValueFrom() or done callback for assertions — subscribe without done silently never asserts

- Test both happy path (success action dispatched) and error path (error action dispatched) for every Effect

- Test the 'stays alive after error' invariant — emit a second action after the error path and assert it still produces output

Weak answers miss: The ReplaySubject vs Subject distinction — most candidates know to use provideMockActions but can't explain why ReplaySubject is required.

Level: Mid

Question 2

"What is selector.projector() and why should you use it instead of testing selectors through MockStore?"

Strong answer covers:

- projector() calls the selector's pure derivation function directly with mock inputs — no store, no TestBed

- Selectors are composed pure functions — testing the projector tests derivation logic in isolation

- Testing via MockStore conflates selector registration with derivation logic — slower, harder to debug

- projector() inputs are the outputs of the selector's input selectors — for selectAdminUsers(selectAllUsers), the projector input is User[], not UsersState

- Faster, simpler, more focused — the gold standard for selector testing

Weak answers miss: What the projector inputs are — candidates often assume the projector takes raw state when it takes the outputs of upstream selectors.

Level: Mid

Section 8 — Quick Reference Card

3-line definition: NgRx Testing has five distinct layers — each tested independently with its own tool: reducers and selector projectors as plain functions (no Angular needed), Effects with ReplaySubject + provideMockActions, components with MockStore + overrideSelector, and Component Store / SignalStore with the real store class and a mocked service. The golden rule: never assert inside subscribe() without done — Jasmine exits before the callback runs and the test is permanently green regardless of correctness.

Syntax at a glance:

```
// Reducer const result = usersReducer(initialState, UsersApiActions.usersLoaded({ users })); expect(result.users).toEqual(users); expect(result).not.toBe(initialState); // New reference // Selector projector const result = selectAdminUsers.projector(mockUsers); // Input = output of input selectors expect(result).toHaveSize(1); // Component + MockStore providers: [provideMockStore()] store.overrideSelector(selectAllUsers, mockUsers); store.refreshState(); fixture.detectChanges(); afterEach(() => store.resetSelectors()); // ALWAYS // Effect actions$ = new ReplaySubject<Action>(1); // Not Subject providers: [provideMockActions(() => actions$), provideMockStore()] actions$.next(UsersPageActions.pageOpened()); const action = await firstValueFrom(effects.loadUsers$); // Not subscribe without done // HTTP interception (Angular 18+) providers: [provideHttpClient(), provideHttpClientTesting()] afterEach(() => httpController.verify()); // Component Store / SignalStore providers: [DataTableStore, { provide: UserService, useValue: spy }] store.loadUsers(); tick(); // fakeAsync — flush microtasks expect(store.users()).toEqual(mockUsers);
```

Layer	Test tool	Angular needed?	Async handling
Reducer	reducer(state, action)	No	Synchronous
Selector	selector.projector(inputs)	No	Synchronous
Component	MockStore + overrideSelector	Yes — TestBed	detectChanges() after refreshState()
Effect	ReplaySubject + provideMockActions	Yes — TestBed	done / firstValueFrom
Component Store	Real store + mock service	Yes — TestBed	fakeAsync+tick or done+setTimeout
SignalStore	Real store + mock service	Yes — TestBed	fakeAsync+tick or done+setTimeout

When to use: Every NgRx feature needs tests at every layer — reducer, selector, effect, component. End-to-end tests (Playwright/Cypress) cover integration; these unit tests cover each layer in isolation.

The one rule that matters most: Never assert inside subscribe() without a done callback — Jasmine exits before the async callback executes, the assertion never runs, and the test is permanently green regardless of whether the Effect works correctly.

TOPIC 59 — NgRx Best Practices & Architecture Patterns

Consolidated | Difficulty: Advanced | Relevance: Critical

Section 0 — Difficulty & Priority Rating

Difficulty: Advanced — This topic requires you to hold the entire NgRx ecosystem in your head simultaneously and make principled architectural decisions under real-world constraints; the difficulty is not any single new API but the judgment required to apply eight topics of NgRx knowledge correctly at scale.

Angular Relevance: Critical — Every NgRx pattern you have learned in Topics 51-58 can be applied well or badly; this topic is the decision framework that determines which — teams that skip architectural discipline end up with NgRx stores that are harder to maintain than the component state they replaced.

Section 1 — Explanation

The City Planning Analogy

A city needs infrastructure — roads, utilities, zoning laws. A small village can be built informally: houses wherever there's space, roads connecting them ad hoc. That works at small scale.

But as the city grows, informal decisions compound into problems: a road built for ten houses cannot handle ten thousand, a utility line buried without a map cannot be maintained, and a neighbourhood zoned 'whatever fits' becomes a maintenance nightmare.

NgRx architecture is city planning for application state. At small scale, any approach works. At team scale and production scale, the decisions you make about where state lives, how it flows, what gets named what, and where side effects are allowed determine whether your codebase is a well-planned city or an informal sprawl.

The five architectural questions NgRx teams must answer:

- Where does this state live? (global store / SignalStore / Component Store / local signal)
- What gets an action? (events, not commands — naming discipline)
- Where do side effects go? (always Effects, never components)
- How are selectors composed? (layered, memoised, one view model per component)
- How is the store structured? (feature slices, lazy loading, Entity for collections)

State characteristic	Recommended tool
Shared across multiple features	Global NgRx Store
Survives navigation (persistent)	Global NgRx Store
Needs DevTools / time-travel	Global NgRx Store
Collection of identifiable records	NgRx Entity (Topic 54)

State characteristic	Recommended tool
URL-driven state	NgRx Router Store (Topic 57)
Complex local UI state (3+ slices, async)	NgRx SignalStore or Component Store
Simple local UI state (1-2 values)	Angular signal()
One-off flag or toggle	Local signal() in component

Section 2 — Connection to Prior Concepts

Directly builds on every NgRx topic:

- Topic 51 — NgRx Core Concepts: The action naming discipline, reducer purity rules, and one-way data flow are the foundation of every best practice here.
- Topic 52 — NgRx Effects: The operator selection rules (switchMap for reads, concatMap for writes, exhaustMap for submissions) and catchError placement inside inner Observables are non-negotiable production rules.
- Topic 53 — NgRx Selectors In Depth: The layered selector pattern, one view model selector per component, and never returning new literals from projectors are selector architecture rules.
- Topic 54 — NgRx Entity: Entity is the correct structure for any collection of records — avoiding hand-written array CRUD is a best practice, not an option.
- Topics 55 & 56 — Component Store and SignalStore: The boundary between local and global state is the most consequential architectural decision in NgRx.
- Topic 57 — Router Store: The URL-as-state-of-truth principle and Effect-driven data loading are Router Store's contribution to the architecture.
- Topic 58 — NgRx Testing: Testability is an architectural constraint — patterns that make code hard to test are architectural anti-patterns.

Bridging note: In Topic 51, the rules were about correctness — never mutate state, always use createSelector, name actions as past-tense events. In Topic 59, the rules are about architecture — which of these tools belongs in which layer, how features relate to each other, and how the NgRx store grows without becoming unmanageable.

Most directly extends: Topic 51 — NgRx Core Concepts (every best practice is an extension of its foundational discipline).

Section 3 — Code Example

Feature folder structure — the standard layout

```
src/app/features/orders/ ■■■ orders.routes.ts <- Lazy route with provideState +
provideEffects ■■■ orders.actions.ts <- createActionGroup per source ■■■ orders.reducer.ts
<- createFeature + EntityAdapter ■■■ orders.selectors.ts <- Layered selectors + one vm
selector per component ■■■ orders.effects.ts <- All side effects; no HTTP in components ■■■
orders.service.ts <- HTTP only; no store access ■■■ components/ ■ ■■■
orders-list/orders-list.component.ts ■ ■■■ order-detail/order-detail.component.ts ■■■
order.model.ts
```

Actions — one createActionGroup per source, past-tense naming

```
// orders.actions.ts export const OrdersPageActions = createActionGroup({ source: 'Orders
Page', events: { 'Page Opened': emptyProps(), 'Order Selected': props<{ orderId: string }>(),
'Create Order Clicked': props<{ order: Omit<Order, 'id'> }>(), 'Delete Order Clicked':
props<{ orderId: string }>(), }}); export const OrdersApiActions = createActionGroup({
source: 'Orders API', events: { 'Orders Loaded': props<{ orders: Order[] }>(), 'Orders Load
Failed': props<{ error: string }>(), 'Order Created': props<{ order: Order }>(), 'Order
Create Failed': props<{ error: string }>(), 'Order Deleted': props<{ orderId: string }>(),
'Order Delete Failed': props<{ orderId: string; error: string }>(), }}); // NEVER:
command-style names like 'Load Orders', 'Delete Order' // NEVER: generic names like 'Set
Loading', 'Update State'
```

Reducer — createFeature + EntityAdapter + extra slices

```
// orders.reducer.ts export const orderAdapter = createEntityAdapter<Order>({ sortComparer:
(a, b) => new Date(b.createdAt).getTime() - new Date(a.createdAt).getTime(), // Newest first
}); export interface OrdersState extends EntityState<Order> { selectedOrderId: string | null;
loading: boolean; creating: boolean; error: string | null; } const initialState: OrdersState
= orderAdapter.getInitialState({ selectedOrderId: null, loading: false, creating: false,
error: null, }); export const ordersFeature = createFeature({ name: 'orders', reducer:
createReducer(initialState, on(OrdersPageActions.pageOpened, state => ({ ...state, loading:
true, error: null })), on(OrdersPageActions.orderSelected, (state, { orderId }) => ({
...state, selectedOrderId: orderId })), on(OrdersPageActions.createOrderClicked, state => ({
...state, creating: true, error: null })), on(OrdersApiActions.ordersLoaded, (state, { orders
}) => orderAdapter.setAll(orders, { ...state, loading: false }) ),
on(OrdersApiActions.ordersLoadFailed, (state, { error }) => ({ ...state, loading: false,
error })), on(OrdersApiActions.orderCreated, (state, { order }) => orderAdapter.addOne(order,
{ ...state, creating: false }) ), on(OrdersPageActions.deleteOrderClicked, (state, { orderId
}) => orderAdapter.removeOne(orderId, state)), on(OrdersApiActions.orderDeleteFailed, (state,
{ error }) => ({ ...state, error })), ), });
```

Selectors — layered composition, one view model per component

```
// orders.selectors.ts const { selectAll: selectAllOrders, selectEntities:
selectOrderEntities, selectTotal: selectOrderTotal } =
orderAdapter.getSelectors(selectOrdersState); const { selectRouteParams } =
getRouterSelectors(); export const selectOrderIdFromRoute = createSelector(selectRouteParams,
params => params['orderId'] as string | undefined); // Layer 4: View model selectors — one per
component export const selectOrdersListVm = createSelector( selectAllOrders, selectLoading,
selectCreating, selectError, selectOrderTotal, (orders, loading, creating, error, total) =>
({ orders, loading, creating, error, total, isEmpty: total === 0 && !loading, }) ); export
const selectOrderDetailVm = createSelector( selectCurrentRouteOrder, selectLoading,
selectError, (order, loading, error) => ({ order, loading, error }) );
```

Effects — all side effects, correct operator selection

```
// orders.effects.ts @Injectable() export class OrdersEffects { readonly #actions$ =
inject(Actions); readonly #store = inject(Store); readonly #ordersService =
inject(OrdersService); // READ — switchMap: cancel previous load if page opens twice
loadOrders$ = createEffect(() => this.#actions$.pipe( ofType(OrdersPageActions.pageOpened),
switchMap(() => // switchMap for reads this.#ordersService.getAll().pipe( map(orders =>
OrdersApiActions.ordersLoaded({ orders })), catchError(error =>
of(OrdersApiActions.ordersLoadFailed({ error: error.message })))) // catchError INSIDE
switchMap — never on outer stream ) ) ) ); // WRITE — concatMap: queue creates, never cancel
in-flight writes createOrder$ = createEffect(() => this.#actions$.pipe(
ofType(OrdersPageActions.createOrderClicked), concatMap(({ order }) => // concatMap for
ordered writes this.#ordersService.create(order).pipe( map(created =>
OrdersApiActions.orderCreated({ order: created })), catchError(error =>
of(OrdersApiActions.orderCreateFailed({ error: error.message })))) ) ) ); //
NAVIGATION-DRIVEN LOAD — Router Store pattern (Topic 57) loadOrderOnNavigation$ =
createEffect(() => this.#actions$.pipe( ofType(routerNavigatedAction), concatLatestFrom(() =>
this.#store.select(selectOrderIdFromRoute)), filter(([, orderId]) => orderId !== undefined),
switchMap(([, orderId]) => this.#ordersService.getById(orderId!).pipe( map(order =>
OrdersApiActions.ordersLoaded({ orders: [order] })), catchError(error =>
of(OrdersApiActions.ordersLoadFailed({ error: error.message })))) ) ) ); }
```

Lazy route registration

```
// orders.routes.ts export const ORDERS_ROUTES: Routes = [{ path: '', canActivate:
[authGuard], providers: [ provideState(ordersFeature), // Lazy — loads only when route
activates provideEffects(OrdersEffects), ], children: [ { path: '', loadComponent: () =>
import('./components/orders-list/orders-list.component').then(m => m.OrdersListComponent) },
{ path: ':orderId', loadComponent: () =>
import('./components/order-detail/order-detail.component').then(m => m.OrderDetailComponent)
}, ], }];
```

Component — thin view layer, reads from store only

```
@Component({ selector: 'app-orders-list', standalone: true, changeDetection:
ChangeDetectionStrategy.OnPush, // Always with NgRx template: ` @let vm =
store.selectSignal(selectOrdersListVm)(); @if (vm.loading) { <app-spinner /> } @if (vm.error)
{ <app-error [message]="vm.error" /> } @if (vm.isEmpty) { <p>No orders yet.</p> } @for (order
of vm.orders; track order.id) { <app-order-card [order]="order" (delete)="onDelete(order.id)"
(select)="onSelect(order.id)" /> } <p>{{ vm.total }} orders total</p> ` }) export class
OrdersListComponent implements OnInit { readonly store = inject(Store); // public readonly —
template access protected readonly selectOrdersListVm = selectOrdersListVm; ngOnInit(): void
{ this.store.dispatch(OrdersPageActions.pageOpened()); } onSelect(id: string): void {
this.store.dispatch(OrdersPageActions.orderSelected({ orderId: id })); } onDelete(id:
string): void { this.store.dispatch(OrdersPageActions.deleteOrderClicked({ orderId: id })); }
}
```

ESLint enforcement — mechanical rule checking

```
// .eslintrc.json { "rules": { "@ngrx/no-dispatch-in-effects": "error",
"@ngrx/no-effects-in-providers": "error", "@ngrx/prefer-action-creator": "error",
"@ngrx/prefer-selector-in-select": "error", "@ngrx/prefer-concat-latest-from": "error",
"@ngrx/avoid-combining-component-store-selectors": "warn" } }
```

Section 4 — Before & After Refactor

Before — Mixed-concern component with direct HTTP and hand-rolled actions

```
// WRONG: component making HTTP calls directly alongside NgRx @Component({}) export class
OrdersComponent implements OnInit { readonly #store = inject(Store); readonly #ordersService
= inject(OrdersService); // Service in component orders =
this.#store.selectSignal(selectAllOrders); ngOnInit(): void {
this.#ordersService.getAll().subscribe(orders => {
this.#store.dispatch(OrdersApiActions.ordersLoaded({ orders })); // No error handling, no
cancellation, no DevTools visibility }); } delete(id: string): void { this.#store.dispatch({
type: 'DELETE_ORDER', id } as any); // Hand-rolled, command-style } }
```

What breaks: Component owns HTTP logic — untestable without real HTTP. No DevTools visibility. Hand-rolled actions bypass TypeScript safety. No cancellation semantics — rapid navigation triggers multiple parallel loads.

After — Thin view layer, past-tense events, Effect-driven

```
// CORRECT: Component is a thin view layer @Component({ standalone: true, changeDetection:
ChangeDetectionStrategy.OnPush }) export class OrdersListComponent implements OnInit {
readonly #store = inject(Store); readonly orderVm =
this.#store.selectSignal(selectOrdersListVm); ngOnInit(): void {
this.#store.dispatch(OrdersPageActions.pageOpened()); } delete(id: string): void {
this.#store.dispatch(OrdersPageActions.deleteOrderClicked({ orderId: id })); } }
```

Why: Component has zero HTTP dependency — testable with only MockStore. DevTools shows the complete chain: [Orders Page] Page Opened -> [Orders API] Orders Loaded. Cancellation and error handling are the Effect's responsibility — consistent app-wide.

Section 5 — Common Mistakes

Mistake 1 — Putting ephemeral UI state in the global store

```
// WRONG: ephemeral UI state in the global store export const appReducer = createReducer({
isNavMenuOpen: false, // Local UI toggle — should NOT be here activeTabIndex: 0, // Local tab
state modalOpen: false, // Local modal flag formStep: 1, // Local wizard step // These DO
belong here: currentUser: null, orders: [], notifications: [], }); // CORRECT: local signals
for ephemeral UI state @Component({}) export class NavComponent { isMenuOpen = signal(false);
// Lives and dies with this component activeTab = signal(0); }
```

Angular consequence: The global store becomes a dumping ground. DevTools logs thousands of [Nav] Menu Opened actions hiding meaningful changes. Reducers and selectors multiply for state that never needed to be shared.

Mistake 2 — Writing command-style actions instead of event-style actions

```
// WRONG: Commands — describe what to do (implementation detail) dispatch({ type:
'LOAD_ORDERS' }) dispatch({ type: 'SET_LOADING', loading: true }) dispatch({ type:
'UPDATE_SELECTED_ORDER', orderId }) // CORRECT: Events — describe what happened (facts)
dispatch(OrdersPageActions.pageOpened()) dispatch(OrdersApiActions.ordersLoaded({ orders }))
dispatch(OrdersPageActions.orderSelected({ orderId }))
```

Angular consequence: Tight coupling between the component (which now knows what effect to trigger) and the effect (which is supposed to own that knowledge). One event can be handled by multiple reducers and effects simultaneously — command-style actions make this impossible.

Mistake 3 — Using switchMap for write operations

```
// WRONG: switchMap for DELETE — cancels in-flight delete on double-click deleteOrder$ =
createEffect(() => this.#actions$.pipe( ofType(OrdersPageActions.deleteOrderClicked),
switchMap(({ orderId }) => // WRONG for writes this.#ordersService.delete(orderId).pipe(...)
) ) ); // CORRECT: concatMap for ordered writes — never cancels deleteOrder$ = createEffect(()
=> this.#actions$.pipe( ofType(OrdersPageActions.deleteOrderClicked), concatMap(({ orderId })
=> // Queue deletes in order this.#ordersService.delete(orderId).pipe(...) ) ) );
```

Angular consequence: Data corruption — a user double-clicks delete, the first DELETE request is cancelled mid-flight, the server never deletes the order, the UI shows it deleted, the next page load restores it. No error, no console warning.

Section 6 — Do's and Don'ts

DO	DON'T
Use createActionGroup with one group per source — typed, discoverable, consistent	Hand-roll action objects with { type: 'LOAD_ORDERS' } — no TypeScript safety
Name actions as past-tense events: [Orders Page] Order Delete Clicked	Name actions as commands: Load Orders, Set Loading
Put ALL side effects in Effects — HTTP, navigation, analytics, timers	Call services directly in components alongside store.dispatch()
Use concatMap for writes (DELETE, POST, PUT) that must not be cancelled	Use switchMap for write operations — silently cancels in-flight write requests
Register feature state lazily with provideState() in route providers	Register all feature state in app.config.ts — defeats lazy loading
Apply ChangeDetectionStrategy.OnPush to every NgRx-connected component	Leave components on default change detection — every store action triggers CD on entire tree
Use @ngrx/eslint-plugin to enforce rules mechanically	Rely solely on code review for architectural discipline

Section 7 — Interview Prep

Question 1

"How do you decide whether state belongs in the global NgRx store, NgRx SignalStore, or a local Angular Signal?"

Strong answer covers:

- Global NgRx store: shared across multiple features, persists across navigation, needs DevTools audit trail, or drives complex async Effect chains
- SignalStore or Component Store: complex local feature state with 3+ slices or async side effects, scoped to one component and its children, destroyed when component is destroyed
- Local signal(): simple 1-2 value UI state (toggle, tab index, modal open) that nothing outside the component needs
- The decision question: 'Does anything outside this component need this state, today or in the foreseeable future?'

- Practical rule: start local, promote to global only when sharing pain is felt

Weak answers miss: The SignalStore middle ground — most candidates know global NgRx vs local component state but miss that SignalStore covers the large middle territory between them.

Level: Mid

Question 2

"What is the difference between an action as an event versus an action as a command, and why does it matter architecturally?"

Strong answer covers:

- Event: describes what happened ([Orders Page] Order Delete Clicked) — past tense, fact-based, source-named
- Command: describes what to do ([Orders] Delete Order) — imperative, implementation-coupled
- Events decouple the dispatcher (component) from the handler (reducer/effect) — the component doesn't know or care what will happen as a result
- One event can be handled by multiple reducers AND multiple effects simultaneously — impossible with command-style actions without creating many fine-grained commands
- DevTools readability: filtering by [Orders Page] shows every user interaction; filtering by [Orders API] shows every server response

Weak answers miss: The 'one event, multiple handlers' architectural benefit — most candidates know the naming convention but can't articulate why it enables better architecture.

Level: Senior

Section 8 — Quick Reference Card

3-line definition: NgRx best practices are seven laws — state in the right layer, actions as past-tense events, all side effects in Effects, correct operator selection for each write pattern, catchError inside inner Observables, feature state lazy-registered, and OnPush on every connected component. Every production NgRx bug traces back to violating exactly one of these. The most consequential decision is state placement: global store for shared/persistent, SignalStore for complex local, signal() for simple local — never everything in the global store.

The seven laws:

Law	Rule
1. State placement	Global store for shared/persistent; SignalStore for complex local; signal() for ephemeral
2. Action naming	Past-tense events with [Source] Event Name via createActionGroup — never commands
3. Effect ownership	All side effects in Effects — components dispatch and read, nothing else
4. Operator selection	switchMap = reads only; concatMap = ordered writes; exhaustMap = submissions

Law	Rule
5. Error handling	catchError inside inner Observable — on outer stream it permanently kills the Effect
6. State registration	Lazy via provideState() in route providers — never eager in app.config.ts
7. Change detection	OnPush on every NgRx-connected component — mandatory, not optional

Decision matrix:

State type	Correct tool
Shared across 3+ features	Global NgRx Store
Collection of records	NgRx Entity
Navigation-driven data	Router Store + Effect
Complex local UI (3+ slices)	SignalStore
Simple toggle / flag	Local signal()
Auth state (app-wide)	Global NgRx Store
HTTP side effects	NgRx Effects

When to use NgRx: Multiple features share state; team has 3+ developers; state changes need audit trails; complex async sequences (dependent HTTP, real-time, optimistic mutations).

When not to use NgRx: Single-developer project or prototype; state is primarily local to individual pages; team is not already familiar with RxJS; content-heavy app with minimal user interaction.

The one rule that matters most: Never use switchMap for write operations (POST, PUT, DELETE) — it silently cancels in-flight requests on any new emission, causing data corruption with no error message and no console warning.
